به نام خدا

جزوه جامع آموزشی درس مهندسی نرم‌افزار

مرجع: کتاب مهندسی نرم‌افزار پرسمن *ویرایش ۹* (Roger S. Pressman & Bruce R. Maxim)

مدرس: **محمدعلی رسول‌زاده**

M.A. Rasoulzadeh

پائیز ۱۴۰۵

نسخه ۱٫۰

فهرست مطالب

فصل اول: ماهیت نرم‌افزار و مهندسی نرم‌افزار (Software & Software Engineering) ۳

فصل دوم: مدل‌های فرآیند نرم‌افزار (Process Models) ۹

فصل سوم: توسعه چابک، اسکرام، کانبان و DevOps (Agility and Process) ۱۸

فصل هفتم: درک و استخراج نیازمندی‌ها (Understanding Requirements) ۲۵

فصل هشتم: مدلسازی نیازمندی‌ها (Requirements Modeling) ۳۱

فصل نهم: مفاهیم و اصول پایه طراحی نرم‌افزار (Design Concepts) ۳۷

فصل اول: ماهیت نرم‌افزار و مهندسی نرم‌افزار (Software & Software Engineering)

📌 راهنمای مطالعه جزوه: این جزوه به صورت کاملاً تشریحی و کاربردی منطبق بر کتاب پرسمن ویرایش نهم تدوین شده است. تمام مفاهیم تئوری با مثال‌های واقعی از پروژه‌های نرم‌افزاری مختلف همراه شده‌اند تا آمادگی کامل برای آزمون کتبی و پروژه عملی ایجاد شود.

۱. ماهیت نرم‌افزار (The Nature of Software)

امروزه نرم‌افزار نقشی دوگانه (Dual Role) در دنیای مدرن ایفا می‌کند: هم خود یک محصول (Product) است و هم به عنوان مجرای تحویل یک محصول (Vehicle for delivering a product) عمل می‌کند. به عنوان محصول، توان محاسباتی سخت‌افزار را به کار می‌گیرد و به عنوان مجرا، اطلاعات را دریافت، تغییر شکل داده و مدیریت می‌کند.

۱.۱. تعریف دقیق نرم‌افزار (Software Definition)

کتاب پرسمن نرم‌افزار را صرفاً «کدنویسی» نمی‌داند، بلکه نرم‌افزار را ترکیبی از ۳ عنصر بنیادی می‌شناسد:

۱. دستورالعمل‌ها (Instructions / Programs): کدهایی که در صورت اجرا، ویژگی‌ها، عملکردها و کارایی مورد نظر را ارائه می‌دهند.

۲. ساختارهای داده (Data Structures): ساختارهایی که به برنامه اجازه می‌دهند اطلاعات را به طور مناسب دستکاری کنند.

۳. مستندات (Documentation): توصیفات متنی و گرافیکی که نحوه کارکرد و استفاده از برنامه را به کاربران و توسعه‌دهندگان آموزش می‌دهند.

💡 مثال کاربردی ۱ (سامانه پرداخت اینترنتی بانک): در یک درگاه پرداخت بانکی، دستورالعمل‌ها شامل کدهای سیشارپ هستند که تراکنش مالی را اعتبار سنجی می‌کنند؛ ساختار داده‌ها شامل مدل‌های داده‌ای JSON و جداول دیتابیس برای ثبت اطلاعات کارت و شماره پیگیری است؛ و مستندات شامل راهنمای API برای توسعه‌دهندگان وب‌سایت‌های فروشگاهی است. اگر هر یک از این ۳ بخش نباشد، سیستم کامل نیست.

۱.۲. تفاوت‌های کلیدی نرم‌افزار و سخت‌افزار (Software vs Hardware)

پرسمن تفاوت بنیادی نرم‌افزار و سخت‌افزار را در نحوه پیدایش و استهلاک آن‌ها می‌داند:

• نرم‌افزار توسعه می‌یابد یا مهندسی می‌شود؛ به صورت کارخانه‌ای «ساخته» نمی‌شود (Software is developed, not manufactured).

• نرم‌افزار «فرسوده و مستهلک نمی‌شود» اما «دچار اضمحلال و خرابی می‌شود» (Software doesn't wear out, but it deteriorates).

• اکثر نرم‌افزارها کماکان بر اساس سفارشی‌سازی ساخته می‌شوند و استفاده از قطعات پیش‌ساخته (Reusable components) تازه در حال گسترش است.

ویژگی	سخت‌افزار (Hardware)	نرم‌افزار (Software)
نحوه تولید	تولید انبوه در کارخانه با قطعات فیزیکی	توسعه مبتنی بر دانش، تحلیل و کدنویسی
الگوی خرابی	منحنی وان حمام (Bathtub Curve) - استهلاک فیزیکی	منحنی اضمحلال به دلیل تغییرات مداوم (Side effects of changes)

۱.۳. دامنه‌های کاربردی نرم‌افزار (Software Application Domains)

طبق تقسیم‌بندی پرسمن، نرم‌افزارها در ۷ دامنه اصلی گروه‌بندی می‌شوند که هرکدام چالش‌های مهندسی خاص خود را دارند:

۱. نرم‌افزار سیستم (System Software): نرم‌افزارهایی که به سایر نرم‌افزارها خدمت‌رسانی می‌کنند؛ مثل سیستم‌عامل‌ها، درایورها، کامپایلرها و پردازنده‌های مخابراتی. (مثال: سیستم‌عامل لینوکس یا درایور کارت گرافیک)

۲. نرم‌افزار کاربردی (Application Software): برنامه‌های مستقل که نیازهای تجاری یا فنی خاصی را حل می‌کنند. (مثال: سیستم حسابداری همکاران سیستم یا نرم‌افزار مدیریت انبار)

۳. نرم‌افزار مهندسی/علمی (Engineering/Scientific Software): برنامه‌های متمرکز بر محاسبات ریاضی سنگین و الگوریتم‌های پیچیده. (مثال: نرم‌افزار شبیه‌سازی پرواز یا پردازش سیگنال‌های زلزله‌نگاری در MATLAB)

۴. نرم‌افزار نهفته/جاسازی‌شده (Embedded Software): نرم‌افزارهایی که داخل یک سخت‌افزار مقیم شده و آن را کنترل می‌کنند. (مثال: نرم‌افزار کنترل ترمز ABS خودرو یا فریم‌ورک ماشین لباسشویی هوشمند)

۵. نرم‌افزار خط محصول (Product-Line Software): نرم‌افزارهای تجاری که برای استفاده هزاران مشتری با قابلیت تنظیم ساخته می‌شوند. (مثال: مجموعه آفیس یا نرم‌افزار مدیریت فروشگاه فتوشاپ)

۶. برنامه‌های وب و موبایل (Web/Mobile Apps): نرم‌افزارهای شبکه‌محور که روی مرورگر یا گوشی هوشمند اجرا می‌شوند و با سرور در ارتباطند. (مثال: اپلیکیشن اسنپ یا دیجی‌کالا)

۷. نرم‌افزار هوش مصنوعی (AI Software): نرم‌افزارهایی که از الگوریتم‌های یادگیری ماشین و شبکه‌های عصبی برای حل مسائل غیرترتیبی استفاده می‌کنند. (مثال: سیستم تشخیص تومور سرطانی از روی تصویر MRI)

۱.۴. نرم‌افزارهای قدیمی و میراثی (Legacy Software)

نرم‌افزارهای میراثی (Legacy Systems) سیستم‌های قدیمی هستند که دهه‌ها پیش نوشته شده‌اند اما کماکان شریان حیاتی سازمان‌ها را تشکیل می‌دهند. چرا این سیستم‌ها تغییر می‌کنند؟

۱. تطبیق با محیط‌های جدید (مثال: انتقال دیتابیس قدیمی به ابر/Cloud).

۲. برآورده ساختن نیازمندی‌های جدید تجاری (مثال: اضافه کردن پرداخت با رمز ارز به سیستم بانکی قدیمی).

۳. بازطراحی برای تعامل با سایر سیستم‌های مدرن (Web Services / APIs).

۴. بازسازی معماری جهت افزایش ماندگاری و قابلیت نگهداری (Refactoring).

۲. تعریف دیسیپلین مهندسی نرم‌افزار (Defining Software Engineering)

مؤسسه IEEE مهندسی نرم‌افزار را این‌گونه تعریف می‌کند:

«به‌کارگیری یک رویکرد سیستماتیک، انتظام‌یافته و قابل اندازه‌گیری برای توسعه، بهره‌برداری و نگهداری نرم‌افزار؛ یعنی اعمال مهندسی روی نرم‌افزار.»

۲.۱. ساختار لایه‌ای مهندسی نرم‌افزار (Layered Technology)

پرسمن مهندسی نرم‌افزار را به عنوان یک فناوری لایه‌ای (Layered Technology) تصویر می‌کند که شالوده آن «تعهد به کیفیت» است:

۱. ابزارها (Tools): پشتیبانی خودکار یا نیمه‌خودکار برای فرآیندها و روش‌ها (مانند IDEها، Git، SonarQube).

۲. روش‌ها (Methods): چگونگی انجام کارهای فنی مهندسی نرم‌افزار (شامل تحلیل، طراحی، کدنویسی، تست و نگهداری).

۳. فرآیند (Process): چسبی که تمام لایه‌ها را به هم متصل نگه می‌دارد؛ تعیین‌کننده ترتیب گام‌ها، تحویل‌دادنی‌ها و کنترل‌ها.

۴. تمرکز بر کیفیت (Quality Focus): سنگ‌بنا و بستر اصلی تمام لایه‌ها. بدون تعهد به کیفیت، سایر لایه‌ها بی‌ارزش خواهند بود.

۳. فرآیند نرم‌افزار (The Software Process)

فرآیند نرم‌افزار چارچوبی شامل فعالیت‌ها، وظایف و اقدامات لازم برای ساخت نرم‌افزار باکیفیت است. این فرآیند از دو بخش اصلی تشکیل شده است: فعالیت‌های چارچوبی (Framework Activities) و فعالیت‌های چتری (Umbrella Activities).

۳.۱. فعالیت‌های چارچوبی ۵گانه (Framework Activities)

۱. ارتباطات (Communication): جلسه با مشتری و ذینفعان برای استخراج نیازمندی‌ها و تعریف اهداف پروژه. (مثال: مصاحبه با پزشکان برای ساخت سامانه نسخه الکترونیک)

۲. برنامه‌ریزی (Planning): برآورد زمان، هزینه، منابع و ریسک‌ها و چیدن زمان‌بندی (Task scheduling). (مثال: تعیین Sprintها در اسکرام)

۳. مدلسازی (Modeling): تحلیل و طراحی نرم‌افزار (رسم نمودارهای UML، دیاگرام دیتابیس و معماری). (مثال: رسم Sequence Diagram برای احراز هویت)

۴. ساخت (Construction): کدنویسی (Coding) و تست واحد (Testing) جهت اطمینان از رفع باگ‌ها.

۵. استقرار (Deployment): تحویل نرم‌افزار به مشتری، دریافت بازخورد و ارزیابی نهایی نتایج.

۳.۲. فعالیت‌های چتری (Umbrella Activities)

فعالیت‌های چتری مستقل از یک فاز خاص هستند و در تمام طول چرخه حیات نرم‌افزار اجرا می‌شوند تا کیفیت و سلامت پروژه تضمین شود:

• پیگیری و کنترل پروژه (Software Project Tracking & Control): سنجش پیشرفت واقعی پروژه نسبت به برنامه زمان‌بندی.

• مدیریت ریسک (Risk Management): شناسایی و کاهش ریسک‌های فنی و مدیریتی (مثال: قطع شدن سرور دیتابیس در حین توسعه).

• تضمین کیفیت نرم‌افزار (Software Quality Assurance - SQA): بررسی انطباق کارهای انجام شده با استانداردهای تعریف‌شده.

• بازبینی‌های فنی (Technical Reviews): ارزیابی کدهای نوشته شده یا دیاگرام‌ها توسط سایر همکاران (Peer Review).

• سنجش و اندازه‌گیری (Measurement): اندازه‌گیری ابعاد پروژه، نرخ باگ‌ها و بهره‌وری تیم.

• مدیریت پیکربندی نرم‌افزار (Software Configuration Management - SCM): مدیریت تغییرات سورس‌کد و ورژن‌ها با ابزارهایی مثل Git.

• مدیریت قابلیت بازاستفاده (Reusability Management): توسعه کامپوننت‌های قابل استفاده مجدد در پروژه‌های آتی.

• آماده‌سازی و مدیریت محصول (Work Product Preparation & Production): تولید مستندات، راهنماها و فایل‌های نصب.

۴. ممارست در مهندسی نرم‌افزار (Software Engineering Practice)

ممارست (Practice) شامل مجموعه اصول، مفاهیم و روش‌هایی است که یک مهندس نرم‌افزار باید در روزمره کاری خود رعایت کند. پرسمن اصول ۷گانه هوکر (Hooker's ۷ Principles) را به عنوان راهنمای عملی معرفی می‌کند:

۱. علت وجودی پروژه را در نظر داشته باشید (The Reason It All Exists): نرم‌افزار باید ارزش تجاری ایجاد کند و مشکلی از کاربر حل کند. کدی که ارزش ایجاد نکند بی‌فایده است.

۲. ساده نگه‌دارید! (KISS - Keep It Simple, Stupid!): تمام طراحی‌ها و کدها باید تا حد امکان ساده باشند تا نگهداری آنها راحت‌تر باشد.

۳. چشم‌انداز پروژه را حفظ کنید (Maintain the Vision): معماری و هدف اصلی سیستم نباید در طول توسعه دچار انحراف شود.

۴. آنچه می‌سازید، دیگران مصرف خواهند کرد (What You Produce, Others Will Consume): طوری کد بزنید و مستند کنید که نفر بعدی بتواند آن را بفهمد و توسعه دهد.

۵. برای آینده برنامه‌ریزی کنید (Open Yourself to the Future): سیستم را طوری بسازید که در آینده بتوان به راحتی آن را تغییر داد و توسعه داد.

۶. برای بازاستفاده برنامه‌ریزی پیش‌دستانه داشته باشید (Plan Ahead for Reuse): کدها و کامپوننت‌ها را مستقل بنویسید تا در سایر پروژه‌ها هم قابل استفاده باشند.

۷. فکـر کنید! (Think!): قبل از شروع به کدنویسی، مسئله را کاملاً تحلیل کرده و در مورد راه‌حل فکر کنید.

۵. افسانه‌ها و باورهای غلط در توسعه نرم‌افزار (Software Myths)

پرسمن به باورهای غلطی اشاره می‌کند که موجب نادرستی انتظارات مشتریان، مدیران و برنامه‌نویسان می‌شود:

دسته‌بندی	افسانه و باور غلط (Myth)	واقعیت صنعتی (Reality)
افسانه‌های مدیریتی (Management)	«ما کتابچه‌ای پر از استانداردها داریم، پس نیازی به نگرانی درباره کیفیت نیست!»	کتابچه‌ها بدون آموزش عملی و فرآیند زنده، به ندرت توسط توسعه‌دهندگان استفاده می‌شوند.
افسانه‌های مشتری (Customer)	«یک تعریف کلی از اهداف پروژه کافیست؛ جزئیات را در حین کدنویسی اضافه می‌کنیم!»	نیازمندی‌های مبهم و مبهم‌سازی اهداف در ابتدای پروژه باعث شکست فاجعه‌بار و هزینه بالا می‌شود.
افسانه‌های توسعه‌دهنده (Practitioner)	«وقتی برنامه‌ای را نوشتیم و کار کرد، کار ما تمام شده است!»	واقعیت: ۶۰ تا ۸۰ درصد تلاش‌های نرم‌افزاری تازه بعد از تحویل اولیه (در فاز نگهداری) رخ می‌دهد.

۶. سوالات نمونه و نکات کلیدی امتحان (Exam Checklist)

سوال ۱: سه عنصر تشکیل‌دهنده نرم‌افزار طبق تعریف پرسمن را نام برده و توضیح دهید. (پاسخ: دستورالعمل‌ها، ساختارهای داده، مستندات)

سوال ۲: چرا منحنی خرابی نرم‌افزار با سخت‌افزار متفاوت است؟ (پاسخ: سخت‌افزار استهلاک فیزیکی دارد، نرم‌افزار خرابی ناشی از عوارض جانبی تغییرات دارد)

سوال ۳: چهار لایه فناوری مهندسی نرم‌افزار را نام ببرید. (پاسخ: کیفیت، فرآیند، روش‌ها، ابزارها)

سوال ۴: پنج فعالیت چارچوبی (Framework Activities) را نام ببرید. (پاسخ: ارتباطات، برنامه‌ریزی، مدلسازی، ساخت، استقرار)

سوال ۵: سه مورد از فعالیت‌های چتری (Umbrella Activities) را توضیح دهید. (پاسخ: مدیریت ریسک، SCM، تضمین کیفیت SQA)

✨ پایان فصل ۱ – موفق و پیروز باشید ✨

فصل دوم: مدل‌های فرآیند نرم‌افزار (Process Models)

پیش‌گفتار و اهداف آموزشی فصل دوم

توسعه نرم‌افزار بدون داشتن یک فرآیند مشخص و هدایت‌شده، مانند ساختن یک برج چندطبقه بدون نقشه معماری و زمان‌بندی ساخت است. در فصل اول آموختیم که مهندسی نرم‌افزار یک فناوری لایه‌ای است که ستون فقرات و پایه و اساس آن، «نگرش بر فرآیند» (Process Focus) است. در این فصل، به بررسی عمیق ساختار فرآیند، انواع مدل‌های تجویزی (Prescriptive)، تکاملی (Evolutionary)، تخصصی (Specialized)، فرآیند یکپارچه (Unified Process) و مدل‌های فردی/تیمی (PSP/TSP) می‌پردازیم.

هدف این جزوه ارائه تشریح کامل مفاهیم، تحلیل نقاط قوت و ضعف هر مدل، و ارائه مثال‌های کاملاً واقعی و کاربردی از پروژه‌های متنوع صنعت نرم‌افزار است تا دانشجویان بتوانند علاوه بر موفقیت در آزمون‌های آکادمیک، دیدگاهی عمیق و صنعتی نسبت به انتخاب فرآیند مناسب کسب کنند.

۱. ساختار عمومی فرآیند و جریان‌های کاری (Generic Process Framework)

فرآیند نرم‌افزار (Software Process) به عنوان مجموعه‌ای از فعالیت‌ها، اقدام‌ها و وظایف تعریف می‌شود که برای ساخت یک محصول نرم‌افزاری باکیفیت اجرا می‌گردد. طبق تعریف کتاب پرسمن، یک فرآیند عمومی از ۵ فعالیت چارچوبی (Framework Activities) و چندین فعالیت چتری (Umbrella Activities) تشکیل شده است.

۱.۱. فعالیت‌های ۵گانه چارچوب (Framework Activities)

- ارتباطات (Communication): برقراری ارتباط موثر با مشتری و سایر ذینفعان جهت استخراج نیازمندی‌ها و درک اهداف کسب‌وکار.
- برنامه‌ریزی (Planning): تعیین نقشه راه توسعه، تعریف کارهای فنی، تخمین مخاطرات (ریسک‌ها)، تخصیص منابع و زمان‌بندی پروژه.
- مدلسازی (Modeling): ایجاد مدل‌های تحلیلی و طراحی جهت درک بهتر نیازمندی‌ها و معماری نرم‌افزار (شامل تحلیل نیازمندی‌ها و طراحی الگوریتم/دیتابیس).
- ساخت (Construction): ترکیب کدنویسی (Coding) و تست خودکار/دستی (Testing) برای کشف و رفع باگ‌های نرم‌افزاری.
- استقرار (Deployment): تحویل نسخه نهایی یا افزایشی نرم‌افزار به مشتری، دریافت بازخورد و ارائه خدمات پشتیبانی.

۱.۲. فعالیت‌های چتری (Umbrella Activities)

فعالیت‌های چتری در تمام طول چرخه حیات پروژه و به صورت هم‌زمان با فعالیت‌های ۵گانه چارچوب اجرا می‌شوند تا از کیفیت، مدیریت هزینه و کنترل تغییرات اطمینان حاصل شود:

- ردیابی و کنترل پروژه (Software Project Tracking and Control): سنجش ارزیابی پیشرفت واقعی در مقایسه با زمان‌بندی.
- مدیریت ریسک (Risk Management): شناسایی، تحلیل و کاهش اثرات مخاطرات احتمالی پروژه.
- تضمین کیفیت نرم‌افزار (SQA): تضمین رعایت استانداردهای کیفیت در تمام خروجی‌ها.
- بازبینی‌های فنی رسمی (Technical Reviews): ارزیابی کدهای نوشته شده و مدل‌ها توسط همکاران جهت کشف زودهنگام خطاها.
- مدیریت پیکربندی نرم‌افزار (SCM): کنترل نسخه کدهای سورس و مدیریت تغییرات سندها.
- مدیریت قابلیت استفاده مجدد (Reusability Management): تعریف مؤلفه‌های قابل استفاده مجدد در سایر پروژه‌ها.

۱.۳. جریان فرآیند (Process Flow)

جریان فرآیند نحوه سازمان‌دهی و ترتیب زمانی اجرای فعالیت‌های چارچوب را مشخص می‌سازد. به طور کلی ۴ نوع جریان فرآیندی داریم:

- جریان خطی (Linear Flow): اجرای ترتیبِ گام‌به‌گام ۵ فعالیت از ارتباطات تا استقرار بدون بازگشت به عقب.
- جریان تکرار شونده (Iterative Flow): تکرار یک یا چند فعالیت قبل از ورود به فعالیت بعدی جهت اصلاح و تکمیل.
- جریان تکاملی (Evolutionary Flow): اجرای چرخه‌ای فعالیت‌ها به‌طوری‌که هر چرخه به یک نسخه کامل‌تر و پیشرفته‌تر منجر می‌شود.
- جریان موازی (Parallel Flow): اجرای هم‌زمان چند فعالیت چارچوبی (مثلاً مدلسازی یک بخش هم‌زمان با کدنویسی بخش دیگر).

> 💡 مثال کاربردی واقعی: سامانه هوشمند کنتورهای برق صنعتی (Smart Grid Metering)
>
> در ساخت یک سامانه هوشمند برق صنعتی، فعالیت ارتباطات جهت دریافت پروتکل‌های مخابراتی کنتورها انجام می‌شود. سپس جریان موازی شکل می‌گیرد: تیم سخت‌افزار رویکرد خطی را برای درایورها دنبال می‌کند، درحالی‌که تیم نرم‌افزار مدیریت داده‌ها جریان تکاملی را برای پیاده‌سازی داشبوردهای مدیریتی به کار می‌گیرد. هم‌زمان، فعالیت‌های چتری SCM (جهت کنترل نسخه‌های مختلف firmware) اجرا می‌شود.

۲. مدل‌های فرآیندی تجویزی و سنتی (Prescriptive Process Models)

مدل‌های تجویزی، ساختاری مشخص، منظم و پیش‌فرض را برای مهندسی نرم‌افزار ارائه می‌دهند. این مدل‌ها به دنبال برقراری نظم و پیش‌بینی‌پذیری در توسعه هستند.

۲.۱. مدل آبشاری (The Waterfall Model / Classic Life Cycle)

مدل آبشاری قدیمی‌ترین و شناخته‌شده‌ترین مدل مهندسی نرم‌افزار است که فرآیند را به صورت ترتیبی و خطی متمایز می‌سازد: ابتدا تحلیل کامل نیازمندی‌ها، سپس طراحی، کدنویسی، تست و در نهایت استقرار.

- مزایا: ساختار بسیار منظم و شفاف، مستندسازی فوق‌العاده قوی، مناسب برای تیم‌های تازه کار و پروژه‌های با نیازمندی‌های کاملاً ثابت.
- معایب: عدم انعطاف‌پذیری شدید در برابر تغییرات، پدیده «دره مرگ» (مشتری تا انتهای پروژه هیچ نسخه قابل اجرایی نمی‌بیند)، و ریسک بالای شکست در صورت اشتباه در فهم نیازمندی‌های اولیه.

> مثال کاربردی واقعی: سیستم کنترل پرواز و اویونیک هواپیمای مسافربری
>
> در توسعه نرم‌افزار کنترل پرواز (Flight Control System)، تمام استانداردهای ایمنی و نیازمندی‌های سخت‌افزاری از قبل ۱۰۰٪ مشخص و غیرقابل تغییر هستند. به دلیل اهمیت حیاتی عدم وجود باگ و نیاز به مستندسازی استاندارد DO-۱۷۸C، مدل آبشاری سخت‌گیرانه بهترین انتخاب است زیرا هیچ تغییر ناگهانی در طول پروژه رخ نمی‌دهد.

۲.۲. مدل وی (The V-Model)

مدل V واریانتی از مدل آبشاری است که ارتباط مستقیم بین هر فاز از توسعه (ضلع چپ V) و فاز معادل آزمون و تست (ضلع راست V) را نشان می‌دهد. این مدل تاکید دارد که برنامه‌ریزی تست‌ها باید از همان فازهای اولیه تحلیل و طراحی آغاز شود.

- تست واحد (Unit Test): معادل و متناظر با فاز طراحی تفصیلی و کدنویسی.
- تست یکپارچه‌سازی (Integration Test): معادل با فاز طراحی معماری سیستم.
- تست سیستم (System Test): معادل با فاز طراحی مفهومی و ساختار کلی.
- تست پذیرش کاربر (Acceptance Test): معادل با فاز استخراج نیازمندی‌ها.

> 💡 مثال کاربردی واقعی: سیستم پمپ تزریق انسولین هوشمند بیمارستانی (Medical Infusion Pump)
>
> در سیستم پمپ تزریق پزشکی، زمانی که نیازمندی‌ها نگارش می‌شوند، سناریوهای تست پذیرش بیمارستانی طراحی می‌گردد. هنگام طراحی معماری، سناریوی Integration Test ارتباط سنسور و پمپ نوشته می‌شود. این تناظر مستقیم باعث کاهش خطاهای جانی حیاتی می‌شود.

۲.۳. مدل‌های افزایشی (Incremental Process Models)

مدل افزایشی ویژگی‌های مدل آبشاری را به صورت تکرار شونده ترکیب می‌کند. در این مدل، ابتدا یک «محصول پایه» (Core Product) با حداقل امکانات کلیدی ساخته و تحویل مشتری می‌شود، سپس در هر «افزایش» (Increment)، قابلیت‌های جدیدی به سیستم افزوده می‌گردد.

- ویژگی کلیدی: مدیریت بهتر ریسک، تحویل زودهنگام ارزش به مشتری، و بازخوردگیری عملی از کارکرد محصول پایه.

> 💡 مثال کاربردی واقعی: سامانه جامع یکپارچه دانشگاهی (University ERP)
>
> یک دانشگاه نمی‌تواند ۲ سال منتظر سیستم کامل بماند. در فاز اول (Increment ۱)، سیستم ثبت‌نام اول دوره تحویل داده می‌شود (Core Product). در فاز دوم (Increment ۲)، سیستم پرداخت شهریه و مالی اضافه می‌شود. در فاز سوم (Increment ۳)، سامانه ارزشیابی اساتید و تغذیه متصل می‌گردد.

۳. مدل‌های فرآیندی تکاملی (Evolutionary Process Models)

در دنیای واقعی نرم‌افزار، نیازمندی‌ها در طول زمان تکامل می‌یابند و مشتری دقیقاً نمی‌داند چه می‌خواهد. مدل‌های تکاملی به گونه‌ای طراحی شده‌اند که با هر تکرار، نسخه کامل‌تری از نرم‌افزار تولید می‌شود.

۳.۱. مدل نمونه‌سازی (Prototyping Model)

وقتی مشتری اهداف کلی را می‌داند اما نیازمندی‌های تفصیلی، الگوریتم‌ها یا فرم‌های ورود اطلاعات ناواضح است، مدل نمونه‌سازی استفاده می‌شود:

- مراحل اجرا: ارتباط سریع ◄ برنامه‌ریزی سریع ◄ مدلسازی سریع (Quick Design) ◄ ساخت نمونه اولیه (Prototype) ◄ ارزیابی نمونه توسط مشتری و اصلاح نیازمندی‌ها.

- تله‌ها و چالش‌های نمونه‌سازی: ۱) مشتری ممکن است نمونه اولیه کثیف را به عنوان محصول نهایی تلقی کرده و فشار بیاورد تحویل شود. ۲) توسعه‌دهنده ممکن است از الگوریتم‌ها و الگوی کدنویسی ضعیف سریع استفاده کند و بعداً آن را Refactor نکند.

> 💡 مثال کاربردی واقعی: بازطراحی اپلیکیشن موبایل تجارت الکترونیک (E-Commerce Mobile UX)
>
> برای ابداع یک تجربه کاربری جدید خریدار، ابتدا یک Prototype تعاملی سریع (با ابزارهایی مانند Figma یا Flutter) ساخته می‌شود. کاربران واقعی با آن کار می‌کنند و بازخورد می‌دهند. پس از تثبیت رفتار کاربر، کدنویسی اصلی موتور سفارشات انجام می‌گیرد.

۳.۲. مدل حلزونی / اسپیرال (The Spiral Model - اریک بوم)

مدل حلزونی که توسط Barry Boehm معرفی شد، یک مدل فرآیندی تکاملی است که ماهیت تکرار شونده نمونه‌سازی را با جنبه‌های کنترل‌شده و سیستماتیک مدل آبشاری ترکیب می‌کند. ویژگی برجسته و منحصربه‌فرد این مدل، **تمرکز صریح روی تحلیل و ارزیابی ریسک (Risk Analysis)** در هر تکرار است.

چرخه حلزونی شامل ۴ ربع اصلی است:

- ربع اول (تعیین اهداف): شناسایی اهداف چرخه، راهکارهای بدیل و محدودیت‌ها.
- ربع دوم (ارزیابی و کاهش ریسک): تحلیل فنی و مدیریتی ریسک‌ها، ساخت نمونه اولیه جهت رفع ابهامات فنی.
- ربع سوم (مهندسی و ساخت): توسعه، کدنویسی و تست نسخه آن چرخه.
- ربع چهارم (برنامه‌ریزی دور بعدی): ارزیابی خروجی توسط مشتری و برنامه‌ریزی برای چرخه بعدی.

> 💡 مثال کاربردی واقعی: سامانه هدایت و ناوبری خودروهای خودران (Autonomous Driving System)
>
> به دلیل ریسک‌های فوق‌العاده بالا (جانی، الگوریتمی، پردازش تصویر سنگین)، پروژه در ۵ چرخه‌های حلزونی پیش می‌رود. هر چرخه ریسک‌های جدید پردازش لایدار (LiDAR) یا تاخیر شبکه را ارزیابی کرده و پس از اثبات امنیت، وارد مدار توسعه بعدی می‌شود.

۴. مدل‌های فرآیندی تخصصی (Specialized Process Models)

این مدل‌ها زمانی استفاده می‌شوند که یک رویکرد توسعه‌ای بسیار خاص یا تکنولوژی‌محور در پروژه حاکم باشد.

۴.۱. توسعه مؤلفه‌محور (Component-Based Development)

در این مدل، تاکید اصلی بر بازاستفاده (Reuse) از مؤلفه‌های نرم‌افزاری پیش‌ساخته، کتابخانه‌های تجاری (COTS) یا سرویس‌های موجود است. به جای نوشتن کد از صفر، سیستم از طریق مونتاژ و یکپارچه‌سازی مؤلفه‌ها ساخته می‌شود که زمان توسعه و هزینه را به شدت کاهش می‌دهد.

> 💡 مثال کاربردی واقعی: پلتفرم B۲B SaaS مالی با درگاه‌های چندبانکی
>
> به جای نوشتن سیستم احراز هویت، پیامک، درگاه پرداخت و موتور گزارش‌گیری از صفر، تیم توسعه مؤلفه‌های آماده Stripe، Kavenegar، Auth۰ و Chart.js را یکپارچه کرده و تنها منطق اختصاصی کسب‌وکار خود را می‌نویسد.

۴.۲. مدل روش‌های رسمی (Formal Methods Model)

این مدل شامل اثبات ریاضی صحت نرم‌افزار است. با استفاده از زبان‌های مشخصه‌سازی ریاضی (مانند Z یا VDM)، رفتار سیستم به صورت دقیق فرموله شده و عدم وجود خطاهای منطقی اثبات می‌شود. این مدل بسیار گران، زمان‌بر و نیازمند دانش تخصصی ریاضی است.

> 💡 مثال کاربردی واقعی: سیستم‌عامل میکروکنترلر دستگاه پرتو درمانی بیمارستان (Radiation Therapy)
>
> یک دوز اشتباه پرتو ناشی از خطای نرم‌افزاری می‌تواند منجر به فوت بیمار شود (مانند فاجعه تاریخی Therac-۲۵). در اینجا از روش‌های رسمی برای اثبات ریاضی صحت قفل‌های ایمنی نرم‌افزار استفاده می‌شود.

۴.۳. توسعه جنبه‌گرا (Aspect-Oriented Software Development - AOSD)

در سیستم‌های بزرگ، برخی دغدغه‌ها مانند امنیت (Security)، لاگ‌گیری (Logging)، مدیریت تراکنش (Transaction Management) و ثبت خطا در تمام ماژول‌ها پخش می‌شوند (Cross-cutting Concerns). روش جنبه‌گرا این دغدغه‌ها را مستقل کرده و از طریق تکنیک بافتن (Weaving) به کد اصلی متصل می‌کند.

> 💡 مثال کاربردی واقعی: سامانه بانکداری متمرکز (Core Banking)
>
> در سیستم بانکداری، کدهای مربوط به چک کردن دسترسی کاربر و لاگ‌گیری تراکنش در تمام ۵۰۰ متد مالی حضور دارند. با AOSD، کد لاگ‌گیری یک‌بار نوشته شده و به صورت Aspect به تمام متدها بافته می‌شود بدون آنکه کد مالی کثیف شود.

۵. فرآیند یکپارچه (The Unified Process - UP / RUP)

فرآیند یکپارچه (UP) توسط Ivar Jacobson، Grady Booch و James Rumbaugh طراحی شد تا بهترین ویژگی‌های مدل‌های سنتی و چابک را ترکیب کند. UP یک فرآیند **معماری‌محور (Architecture-Centric)**، **نیازمندی‌محور بر پایه Use Case** و **تکرار شونده و افزایشی** است.

۵.۱. فازهای چهارگانه فرآیند یکپارچه (UP Phases)

- فاز آغازین (Inception): تعریف محدوده پروژه، شناسایی ذینفعان اصلی، استخراج Use Caseهای کلیدی و ارزیابی توجیه اقتصادی.
- فاز جزییات‌سازی / تفصیل (Elaboration): تکمیل اکثر نیازمندی‌ها، حذف ریسک‌های معماری و ساخت «خط پایه معماری» (Architectural Baseline).
- فاز ساخت (Construction): پیاده‌سازی و کدنویسی کامل تمامی Use Caseها، انجام تست‌ها و آماده‌سازی نسخه قابل تحویل.
- فاز انتقال (Transition): تحویل نرم‌افزار به کاربران نهایی، آموزش، تست پذیرش و رفع خطاهای گزارش‌شده در محیط واقعی.

> مثال کاربردی واقعی: سامانه ملی مالیات و عوارض یکپارچه کشور
>
> برای این پروژه کلان ملی، فاز Inception محدوده قوانین مالیاتی را مشخص می‌کند. فاز Elaboration معماری پردازش میلیونی تراکنش‌ها را زیر بار اثبات می‌کند. فاز Construction تمام زیرسیستم‌ها را می‌سازد و فاز Transition سامانه را در سازمان امور مالیاتی مستقر می‌کند.

۶. مدل‌های فرآیندی شخصی و تیمی (PSP & TSP)

Watts Humphrey فرآیندهای PSP و TSP را معرفی کرد تا نشان دهد کیفیت نرم‌افزار حاصل انضباط فردی مهندسان و عملکرد درست تیم است.

- فرآیند نرم‌افزاری شخصی (PSP): توسعه‌دهنده به صورت شخصی زمان صرف‌شده، تعداد خطوط کد، و خطاهای رخ‌داده در هر فاز را اندازه‌گیری و ثبت می‌کند تا اندازه پروژه و کیفیت کدهای خود را بهبود بخشد.
- فرآیند نرم‌افزاری تیمی (TSP): تیم‌های مهندسی خودگردان تشکیل می‌شوند، اهداف پروژه را خودشان تعیین می‌کنند، زمان‌بندی را برآورد کرده و کیفیت خروجی را پایش می‌کنند.

> 💡 مثال کاربردی واقعی: تیم توسعه دهنده سیستم‌عامل سرورهای ابری (Cloud OS Kernel)
>
> در پروژه‌های حساس توسعه کرنل سیستم‌عامل، مهندسان با PSP نرخ خطای خود در هر ۱۰۰۰ خط کد را ثبت می‌کنند و با TSP پروژه‌های انتشار نسخه‌های جدید را برنامه‌ریزی می‌نمایند.

۷. جدول مقایسه جامع مدل‌های فرآیندی نرم‌افزار

جدول زیر یک ماتریس تصمیم‌گیری بسیار قوی برای انتخاب بهترین مدل فرآیندی بر اساس ویژگی‌های پروژه ارائه می‌دهد:

مدل فرآیندی	وضوح نیازمندی اولیه	ارزیابی ریسک	انعطاف در تغییر	مشارکت مشتری	مناسب برای
آبشاری (Waterfall)	بسیار بالا (ثابت)	ضعیف / کم	بسیار پایین	ابتدا و انتهای پروژه	پروژه‌های صنعتی و ایمنی با قوانین ثابت
افزایشی (Incremental)	متوسط	متوسط	متوسط	در پایان هر فاز/افزایش	پروژه‌هایی با نیاز به ورود سریع به بازار
نمونه‌سازی (Prototyping)	بسیار پایین (مبهم)	پایین	بسیار بالا	بسیار بالا و مستمر	پروژه‌های سیستم‌های نوآورانه و UX محور
حلزونی (Spiral)	متوسط تا مبهم	فوق‌العاده قوی	بالا	در پایان هر چرخه	پروژه‌های بسیار بزرگ، گران و پرریسک
فرآیند یکپارچه (UP)	متوسط	قوی (در Elaboration)	بالا	مستمر در تمام فازها	پروژه‌های سازمانی بزرگ و معماری‌محور

۸. نمونه سوالات امتحانی تشریحی فصل دوم با پاسخ کلیدی

سوال ۱: پدیده «دره مرگ» در مدل آبشاری چیست و چرا مدل افزایشی این مشکل را حل می‌کند؟

☑ پاسخ کلیدی امتحانی:

در مدل آبشاری، تا زمانی که کل مراحل تحلیل، طراحی، کدنویسی و تست تمام نشود (نزدیک به انتهای پروژه)، هیچ نسخه قابل اجرایی از نرم‌افزار تحویل مشتری داده نمی‌شود. به این فاصله زمانی طولانی که مشتری بازخوردی نمی‌بیند و عدم اطمینان بالاست «دره مرگ» می‌گویند. مدل افزایشی با ارائه یک «محصول پایه» (Core Product) در همان ماه یا فازهای اول، نسخه کارآمدی را به مشتری داده و این ریسک را کاملاً از بین می‌برد.

سوال ۲: نقش و اهمیت ربع دوم در مدل حلزونی (Spiral Model) چیست؟

☑ پاسخ کلیدی امتحانی:

ربع دوم مدل حلزونی اختصاص به «ارزیابی و کاهش ریسک» دارد. در این مرحله، تمامی خطرات فنی، مدیریتی و مالی پروژه شناسایی شده و با ابزارهایی مانند ساخت نمونه‌های اولیه (Prototypes) یا شبیه‌سازی، صحت فرضیات قبل از صرف هزینه‌های سنگین کدنویسی اثبات می‌گردد.

سوال ۳: فازهای چهارگانه فرآیند یکپارچه (UP) را نام برده و هدف فاز Elaboration را توضیح دهید.

☑ پاسخ کلیدی امتحانی:

فازهای ۴گانه عبارتند از: Inception، Elaboration، Construction و Transition. هدف اصلی فاز Elaboration، تکمیل و تثبیت نیازمندی‌ها، حذف ریسک‌های کلیدی معماری و ایجاد «خط پایه معماری» (Architectural Baseline) است به طوری که ریسک‌های اصلی قبل از فاز ساخت سنگین رفع شده باشند.

۹. تکلیف گیتهاب و پروژه کلاسی جلسه دوم

دانشجویان موظفند برای پروژه انتخابی خود، یک فایل جدید به نام PROCESS.md در ریپازیتوری گیتهاب شخصی خود ایجاد کرده و موارد زیر را مستند کنند:

- انتخاب مدل فرآیندی: کدام مدل فرآیندی (آبشاری، افزایشی، نمونه‌سازی یا UP) را برای پروژه خود انتخاب کرده‌اید؟
- توجیه انتخابی: با حداقل ۳ دلیل فنی (بر اساس میزان وضوح نیازمندی‌ها، ریسک و نوع سیستم) انتخاب خود را تحلیل کنید.
- تعریف فازبندی/افزایش‌ها: مراحل توسعه پروژه خود را در قالب حداقل ۳ افزایش (Increment) یا فاز مشخص کنید.

فصل سوم: توسعه چابک، اسکرام، کانبان و DevOps (Agility and Process)

---

۱. پیش‌گفتار و فلسفه توسعه چابک (Agility)

در دهه‌های پایانی قرن بیستم، صنعت نرم‌افزار با بحرانی جدی به نام «شکست پروژه‌های سنتی» مواجه شد. متدولوژی‌های کلاسیک مانند مدل آبشاری (Waterfall) که مستلزم مستندسازی‌های حجیم و تحلیل‌های طولانی‌پیش‌فرض قبل از نوشتن حتی یک خط کد بودند، توانایی پاسخگویی به تغییرات سریع بازار و نیازهای متغیر مشتریان را نداشتند. محصول نهایی که پس از ماه‌ها یا سال‌ها تحویل داده می‌شد، غالباً با نیازهای روز کاربران فرسنگ‌ها فاصله داشت.

فلسفه توسعه چابک (Agile Software Development) به عنوان یک پاسخ ساختارشکنانه متولد شد. چابکی به معنای حذف برنامه‌ریزی نیست، بلکه به معنای پذیرش تغییر به عنوان یک واقعیت اجتناب‌ناپذیر در مهندسی نرم‌افزار و بازطراحی فرآیندها جهت پاسخگویی سریع، انعطاف‌پذیر و کم‌هزینه به این تغییرات است.

۲. مفهوم چابکی و منحنی هزینه تغییرات (Cost of Change Curve)

یکی از مهم‌ترین مفاهیم تئوریک فصل ۳ کتاب پرسمن، بررسی منحنی هزینه تغییرات در طول چرخه حیات نرم‌افزار است:

- رویکرد سنتی: در روش‌های غیرچابک (سنتی)، منحنی هزینه تغییرات به‌صورت نمایی (Exponential) بالا می‌رود. کشف یک اشتباه در نیازمندی‌ها در فاز تست یا پس از استقرار، نیازمند دستکاری دیتابیس، بازنویسی معماری و تست مجدد کل سیستم است که هزینه‌ای تا ۱۰۰ برابر فاز اولیه ایجاد می‌کند.
- رویکرد چابک: در رویکرد چابک (Agile)، با استفاده از تحویل‌های افزایشی کوتاه (Iterative Increments)، تست مداوم و معماری‌های نرم، تلاش می‌شود که منحنی هزینه تغییرات تا حد امکان مسطح (Flat) نگه داشته شود تا اعمال تغییر در انتهای پروژه بار مالی و زمانی خردکننده‌ای ایجاد نکند.

> 💡 مثال کاربردی صنعت ۱: سامانه مدیریت هوشمند ناوگان حمل‌ونقل (Fleet Management System)
>
> فرض کنید در حال توسعه یک سامانه ناوگان برای یک شرکت بزرگ لجستیک هستید. اگر با روش سنتی پیش بروید و پس از ۶ ماه کدنویسی در هفته پایانی، مدیر لجستیک بگوید: «قوانین مالیاتی جدید تغییر کرده و جریمه مصرف سوخت باید بر اساس فرمول جدید لحظه‌ای محاسبه شود»، در روش سنتی باید ۳ ماه کار را تخریب کنید. اما در رویکرد چابک، این ویژگی در اولین اسپرینت دو هفته‌ای شناسایی، پیاده‌سازی و بازخوردگیری می‌شود، به‌طوری که هزینه اعمال آن نزدیک به صفر است.

۳. بیانیه چابک (Agile Manifesto) و اصول ۱۲گانه

در سال ۲۰۰۱، هفده نفر از برجسته‌ترین رهبران نرم‌افزار دنیا در یورتا جمع شدند و بیانیه چابک را تدوین کردند. این بیانیه شامل ۴ ارزش کلیدی است:

- افراد و تعاملات بالاتر از فرآیندها و ابزارها: برقراری ارتباط مستقیم و حل مسائل بین افراد تیم، ارجح بر پیروی کورکورانه از ابزارها و فرآیندهای پیچیده است.
- نرم‌افزار قابل استفاده بالاتر از مستندات جامع: تحویل یک کُد کارآمد و قابل اجرا به کاربر، ارزش بسیار بیشتری نسبت به نوشتن ۵۰ صفحه سند تحلیلی بدون کد دارد.
- مشارکت با مشتری بالاتر از مذاکرات قراردادی: مشتری هم‌پیمان و هم‌کار تیم توسعه است، نه طرف مقابل یک قرارداد حقوقی سخت‌گیرانه.
- پاسخگویی به تغییرات بالاتر از پیروی از یک طرح اولیه: تغییرات در نیازمندی‌ها فرصت است، نه تهدید! تیم باید طوری طراحی شده باشد که از تغییر استقبال کند.

اصول ۱۲گانه کلیدی چابکی (پرسمن):

- ۱. اولویت اول: رضایت مشتری از طریق تحویل سریع و مداوم نرم‌افزار باارزش.
- ۲. پذیرش تغییر: استقبال از تغییر نیازمندی‌ها، حتی در مراحل پایانی توسعه.
- ۳. تحویل متوالی: تحویل نرم‌افزار قابل کار در بازه‌های زمانی کوتاه (از چند هفته تا چند ماه).
- ۴. هم‌زبانی: همکاری روزانه متخصصان کسب‌وکار و توسعه‌دهندگان در طول پروژه.
- ۵. انگیزه‌بخشی: ایجاد پروژه حول افراد انگیزه دار و فراهم کردن محیط و پشتیبانی لازم.
- ۶. تعامل حضوری: موثرترین روش انتقال اطلاعات، گفتگو و تعامل چهره‌به‌چهره است.
- ۷. معیار پیشرفت: نرم‌افزار کارآمد، سنجه اصلی پیشرفت پروژه است.
- ۸. ریتم پایدار: توسعه پایدار؛ ذینفعان و کاربران باید بتوانند ریتم کاری یکنواختی را حفظ کنند.
- ۹. برتری فنی: توجه مداوم به برتری فنی و طراحی خوب، چابکی را افزایش می‌دهد.
- ۱۰. سادگی (Simplicity): سادگی یا «هنر حداکثر کردن کارهای انجام‌نشده» حیاتی است.
- ۱۱. خودسازمان‌دهی: بهترین معماری‌ها و طراحی‌ها از تیم‌های خودسازمان‌ده (Self-Organizing) بیرون می‌آیند.
- ۱۲. بازتاب و تنظیم: تیم در فواصل منظم نحوه اثربخشی خود را بازبینی و رفتار خود را تنظیم می‌کند.

۴. فریم‌ورک اسکرام (Scrum Framework) – ارکان سه‌گانه

اسکرام محبوب‌ترین و پرکاربردترین فریم‌ورک چابک در دنیاست. پرسمن اسکرام را بر پایه ۳ رکن اصلی تشریح می‌کند: نقش‌ها، مصنوعات و رویدادها.

الف) نقش‌های اصلی در اسکرام (Scrum Roles)

- مالک محصول (Product Owner - PO): نماینده واقعی مشتری و کسب‌وکار. او مسئول تعریف نیازمندی‌ها، اولویت‌بندی کارهای سیستم و مدیریت Product Backlog است. PO تصمیم می‌گیرد چه ویژگی‌هایی ارزش تجاری بیشتری دارند.
- اسکرام مستر (Scrum Master - SM): مربی و تسهیل‌کننده تیم اسکرام. او مسئول اطمینان از اجرای صحیح فرآیند اسکرام است و موانع (Impediments) کاری و فنی را از سر راه تیم برمی‌دارد. SM مدیر یا رئیس تیم نیست!
- تیم توسعه (Development Team): یک تیم فرامتقاطع (Cross-functional) بین ۳ تا ۹ نفر شامل برنامه‌نویسان، طراحان و تست‌کننده‌ها که کار واقعی پیاده‌سازی و ساخت نرم‌افزار را انجام می‌دهند.

ب) مصنوعات اسکرام (Scrum Artifacts)

- بک‌لاگ محصول (Product Backlog): لیستی پویا، متمرکز و اولویت‌بندی‌شده از تمام ویژگی‌ها، داستان‌های کاربر، باگ‌ها و نیازمندی‌های سیستم که باید ساخته شوند.
- بک‌لاگ اسپرینت (Sprint Backlog): مجموعه‌ای از آیتم‌های انتخابی از بک‌لاگ محصول به همراه برنامه پیاده‌سازی آن‌ها برای اجرای در اسپرینت جاری.
- افزوده‌نرم‌افزار (Increment): نسخه کاملاً تست‌شده، سالم و قابل کارکرد نرم‌افزار در انتهای هر اسپرینت که آماده تحویل یا انتشار است.

ج) رویدادهای اسکرام (Scrum Events / Ceremonies)

- ۱. اسپرینت (Sprint): بازه زمانی ثابت (معمولاً ۲ تا ۴ هفته) که در آن یک خروجی ارزشمند نرم‌افزاری ساخته می‌شود.
- ۲. برنامه‌ریزی اسپرینت (Sprint Planning): جلسه شروع اسپرینت برای تعیین اینکه چه کارهایی در این اسپرینت انجام شود و چگونه پیاده‌سازی گردند.
- ۳. اسکرام روزانه (Daily Scrum / Standup): جلسه ۱۵ دقیقه‌ای ایستاده در اول روز. هر عضو به ۳ سوال پاسخ می‌دهد: دیروز چه کردم؟ امروز چه می‌کنم؟ چه مانعی سر راهم است؟
- ۴. بازبینی اسپرینت (Sprint Review): جلسه دمو در انتهای اسپرینت که خروجی قابل کار نرم‌افزار به مالک محصول و ذینفعان نشان داده می‌شود.
- ۵. بازنگری اسپرینت (Sprint Retrospective): جلسه بازنگری درون‌تیمی پس از Review؛ تیم بررسی می‌کند چه چیزهایی خوب پیش رفت و چطور فرآیند کار در اسپرینت بعدی بهتر شود.

> **مثال کاربردی صنعت ۲: پیاده‌سازی سیستم مدیریت امانات و رزرو تجهیزات آزمایشگاه**
>
> در پروژه سیستم آزمایشگاه: استاد یا مدیر آزمایشگاه نقش Product Owner را دارد و اولویت کارهای سیستم را مشخص می‌کند (مثلاً اولویت ۱: ثبت رزرو قطعه، اولویت ۲: محاسبه جریمه دیرکرد). یکی از بچه‌های تیم نقش Scrum Master را بر عهده می‌گیرد تا جلسات ۱۵ دقیقه‌ای روزانه را مدیریت کند. در پایان اسپرینت دو هفته‌ای، اولین خروجی که یک ثبت‌نام ساده و منوی کنسول است دمو می‌شود.

۵. داستان‌های کاربر (User Stories)، معیارهای INVEST و بدهی فنی

در متدولوژی‌های چابک، به جای نوشتن اسناد نیازمندی طولانی، نیازمندی‌ها در قالب «داستان کاربر» نوشته می‌شوند.

فرمت استاندارد نگارش داستان کاربر:

«به عنوان [نقش کاربر]، می‌خواهم [قابلیت/خواسته]، تا بتوانم [ارزش/هدف تجاری].»

معیارهای ۶گانه INVEST برای یک User Story خوب:

- Independent (مستقل): داستان‌ها نباید به یکدیگر وابستگی شدید داشته باشند تا بتوان آن‌ها را مستقلاً پیاده کرد.
- Negotiable (قابل مذاکره): جزئیات داستان جزو خط‌قرمز ثابت نیست و قابل گفتگو بین PO و تیم است.
- Valuable (باارزش): باید یک ارزش واضح برای کاربر یا سیستم ایجاد کند.
- Estimable (قابل تخمین): حجم کار داستان باید توسط تیم قابل تخمین زدن باشد.
- Small (کوچک): اندازه داستان باید آن‌قدر کوچک باشد که در یک اسپرینت جا شود.
- Testable (قابل تست): باید معیارهای پذیرش (Acceptance Criteria) مشخص برای تست شدن داشته باشد.

مفهوم بدهی فنی (Technical Debt)

بدهی فنی استعاره‌ای فوق‌العاده مهم در مهندسی نرم‌افزار است. وقتی تیم برای رسیدن به ددلاین یا تحویل سریع، دست به «کدنویسی کثیف، عدم نوشتن تست، هاردکد کردن متغیرها یا میان‌برزدن در معماری» می‌زند، مثل این است که وام بانکی گرفته است! سود این وام تحویل سریع امروز است، اما اصل و بهره آن (Technical Debt) باید در آینده با صرف زمان زیاد، بازنویسی (Refactoring) و رفع باگ‌های پیچیده پرداخت شود.

> 💡 **مثال کاربردی صنعت ۳: بدهی فنی در سیستم محاسبه جریمه دیرکرد**
>
> اگر برنامه‌نویس نرخ جریمه تاخیر تحویل قطعه را مستقیماً داخل کد هاردکد کند (مثلاً int fine = delay * ۵۰۰۰۰)، این کار سریع انجام می‌شود اما بدهی فنی ایجاد کرده است. هفته بعد که مدیر بگویید نرخ جریمه اساتید با دانشجویان متفاوت است یا روزهای تعطیل نباید حساب شود، کد درهم می‌ریزد! راه درست بازپرداخت بدهی فنی، استفاده از الگوی طراحی Strategy و ساخت سرویس مجزا برای محاسبه جریمه است.

۶. روش کانبان (Kanban) و حد کارهای در حال جریان (WIP Limits)

کانبان یک سیستم مدیریت دیداری جریان کار است که ریشه در خطوط تولید تویوتا در ژاپن دارد و وارد نرم‌افزار شده است. بر خلاف اسکرام که بر بازه‌های زمانی ثابت (Sprint) متمرکز است، کانبان بر «جریان مداوم (Continuous Flow)» تمرکز دارد.

اصول اصلی کانبان:

- ۱. بصری‌سازی جریان کار: استفاده از بورد کانبان شامل ستون‌های To Do (کارهای آینده)، In Progress (در حال انجام) و Done (تکمیل‌شده).
- ۲. محدود کردن WIP (Work-In-Progress Limits): مهم‌ترین اصل کانبان! تعیین حداکثر تعداد کارهای مجاز در ستون In Progress (مثلاً حداکثر ۲ کار همزمان).
- ۳. مدیریت و بهینه‌سازی جریان: جلوگیری از انباشت کارها و گلوگاه‌ها (Bottlenecks) در سیستم.

> 💡 چرا حد WIP حیاتی است؟
>
> اگر ۲ برنامه‌نویس در تیم دارید و حد WIP ستون In Progress را برابر ۲ بگذارید، وقتی ۲ کار در حال انجام است هیچ‌کس حق ندارد کار سومی را شروع کند! همه باید کمک کنند کارهای قبلی Done شوند. این کار مانع از سوییچ کردن مدام ذهن بین کارها (Context Switching) و سوخت‌وساز تیم می‌شود.

۷. رویکرد DevOps و اتوماسیون CI/CD

دیوآپس (DevOps) نام واژه‌ای ترکیبی از Development (توسعه) و Operations (عملیات) است. در شرکت‌های قدیمی، برنامه‌نویسان کد را می‌نوشتند و روی دیوار تیم عملیات/سرور می‌انداختند! اگر سرور می‌خوابید یا کد روی سرور اجرا نمی‌شد، طرفین یکدیگر را متهم می‌کردند.

DevOps این دیوار را فرومی‌ریزد و با ترکیب فرهنگ کار تیمی و ابزارهای اتوماسیون، فرآیند ساخت، تست و استقرار نرم‌افزار را پیوسته می‌کند.

خط‌لوله اتوماتیک CI/CD:

- ادغام مداوم (Continuous Integration - CI): توسعه‌دهندگان کدهای خود را روزانه چند بار در ریپازیتوری مرکزی (مثل GitHub) کامیت و Merge می‌کنند. سیستم به‌صورت خودکار کدها را بیلد کرده و تست‌های واحد را اجرا می‌کند تا خطاهای ادغام سریع کشف شوند.
- تحویل/استقرار مداوم (Continuous Delivery / Deployment - CD): نرم‌افزاری که تست‌های CI را با موفقیت پاس کرده، به‌صورت خودکار آماده انتشار روی سرورهای عملیاتی یا عملیات تست نهایی می‌شود.

۸. جدول مقایسه جامع مدل‌های فرآیندی (پرسمن)

جدول زیر خلاصه مقایسه‌ای بین روش‌های سنتی، اسکرام، کانبان و DevOps است که از مباحث پرتکرار امتحانی می‌باشد:

ویژگی / مدل	سنتی (Waterfall)	اسکرام (Scrum)	کانبان (Kanban)	DevOps
بازه زمانی کار	فازهای طولانی چندماهه	اسپرینت‌های ۲ تا ۴ هفته‌ای	جریان مداوم (بدون اسپرینت)	استقرار لحظه‌ای/روزانه
مدیریت تغییرات	سخت و با هزینه بالا	پذیرش تغییر در انتهای اسپرینت	پذیرش تغییر در هر لحظه	پیوسته و اتوماتیک
معیار موفقیت	تطابق با سند اول ترم	نرم‌افزار قابل استفاده در دمو	بهینه‌سازی سرعت جریان کار	پایداری و سرعت استقرار
تمرکز اصلی	مستندات و فرآیند	تیم خودسازمان‌ده و نقش‌ها	بصری‌سازی و حد WIP	اتوماسیون بیلد، تست و انتشار
تعامل مشتری	فقط اول و انتهای پروژه	مداوم در پایان هر اسپرینت	مداوم و بر اساس اولویت	بازخورد لحظه‌ای از سرور

۹. نمونه سوالات امتحانی تشریحی و پاسخ‌های کلیدی

سوال ۱: چرا منحنی هزینه تغییرات در روش‌های چابک مسطح (Flat) است اما در روش‌های سنتی نمایی است؟

پاسخ: در روش‌های سنتی به دلیل عدم حضور مشتری در طول پروژه و تست دیرهنگام در انتهای کار، کشف یک خطا نیازمند تغییر در معماری و دیتابیس و تست مجدد کل سیستم است که هزینه نمایی دارد. اما در روش‌های چابک به دلیل اسپرینت‌های کوتاه ۲ هفته‌ای، تست‌های خودکار و بازخوردگیری مداوم از مشتری، تغییرات در بازه‌های کوچک شناسایی شده و با کمترین تغییر در کد اعمال می‌شوند.

سوال ۲: ارکان ۳گانه اسکرام را نام برده و نقش Product Owner را با یک مثال توضیح دهید.

پاسخ: ارکان ۳گانه اسکرام عبارتند از: ۱. نقش‌ها (PO, SM, Dev Team) ۲. مصنوعات (Product Backlog, Sprint Backlog, Increment) ۳. رویدادها (Sprint, Planning, Standup, Review, Retro). نقش Product Owner نماینده مشتری است که اولویت کارهای بک‌لاگ را تعیین می‌کند. مثلاً در سامانه تاکسی اینترنتی، PO تصمیم می‌گیرد که ویژگی «پرداخت آنلاین» اولویت بالاتری نسبت به «امتیازدهی به راننده» دارد.

سوال ۳: مفهوم بدهی فنی (Technical Debt) چیست و چه خطراتی برای پروژه دارد؟

پاسخ: بدهی فنی زمانی ایجاد می‌شود که تیم برای تحویل سریع دست به کدنویسی کثیف، میان‌برزدن در معماری یا حذف تست‌ها می‌زند. خطرات آن شامل کاهش شدید سرعت توسعه در آینده، سخت شدن توسعه ویژگی‌های جدید و افزایش باگ‌های ناخواسته در سیستم است که باید با عملیات بازنویسی (Refactoring) بازپرداخت شود.

سوال ۴: هدف از اعمال محدودیت کار در جریان (WIP Limit) در کانبان چیست؟

پاسخ: محدودیت WIP حداکثر تعداد کارهای مجاز در حال انجام در بورد کانبان را تعیین می‌کند. هدف آن جلوگیری از سوییچ کردن مدام ذهن برنامه‌نویسان بین کارهای مختلف، کشف گلوگاه‌ها در سیستم و اطمینان از به سرانجام رسیدن کارهای قبلی قبل از شروع کارهای جدید است.

۱۰. راهنمای تکلیف گیت‌هاب و تمرین منزل (ساخت AGILE.md)

دانشجویان محترم باید برای پروژه انتخابی خود، فایلی به نام `AGILE.md` در ریپازیتوری گیت‌هاب پروژه بسازند و موارد زیر را کامیت کنند:

- بخش اول: تعریف نقش‌های اعضای تیم (مشخص کردن PO و Scrum Master و اعضای توسعه).
- بخش دوم: نگارش حداقل ۵ داستان کاربر (User Story) با فرمت استاندارد `As a... I want... So that...`.
- بخش سوم: اولویت‌بندی داستان‌ها در قالب یک Product Backlog اولیه.
- بخش چهارم: تعریف ۲ داستان کاربر برای اولین اسپرینت دو هفته‌ای (Sprint ۱ Backlog).
- بخش پنجم: ذکر یک نمونه بدهی فنی فرضی در پروژه و ارائه راهکار Refactoring برای آن.

فصل هفتم: درک و استخراج نیازمندی‌ها (Understanding Requirements)

> 💡 پیش‌گفتار و اهداف آموزشی فصل هفتم
>
> این جزوه بر اساس مرجع استاندارد مهندسی نرم‌افزار (پرسمن و مکسیم) جهت تسلط کامل اساتید و دانشجویان تدوین شده است.
>
> در این فصل یاد می‌گیریم که چطور پیش از شروع کدنویسی، خواسته واقعی مشتری را شفاف‌سازی کرده، ذینفعان را شناسایی کنیم، نیازمندی‌های عملکردی و غیرعملکردی را تفکیک نماییم و از بروز اشتباهات ۱۰۰ برابری در فازهای بعدی جلوگیری کنیم.

۱. چرا درک نیازمندی‌ها حیاتی‌ترین فاز مهندسی نرم‌افزار است؟

یکی از بزرگ‌ترین دلایل شکست پروژه‌های نرم‌افزاری در دنیا، «ساختن محصول اشتباه» است. اگر یک مهندس نرم‌افزار بدون استخراج دقیق نیازمندی‌ها شروع به کدنویسی کند، مانند معماری است که بدون نقشه و بدون صحبت با صاحب‌خانه، شروع به گودبرداری و آجرچینی کند! در مهندسی نرم‌افزار، هزینه اصلاح یک خطای نیازمندی اگر در فاز استخراج کشف شود ۱ واحد است، اما اگر همین خطا پس از تحویل نرم‌افزار به مشتری کشف شود، هزینه اصلاح آن ۱۰۰ برابر خواهد بود.

> 💡 مثال واقعی از صنعت: سامانه پذیرش و پرونده الکترونیک بیمارستانی (Hospital ERP)
>
> در یک پروژه بیمارستانی، پزشکان خواستار ثبت سریع نسخه‌ها بودند. اگر تیم نرم‌افزار بدون تحلیل نیازمندی‌ها فرم‌های متعددی با ۲۰ فیلد اجباری می‌ساخت، پزشکان در شرایط اورژانسی سیستم را کنار می‌گذاشتند. مهندسی نیازمندی‌ها نشان داد که نیازمندی اصلی «سرعت ورود داده با حداکثر ۳ کلیک» است، نه فقط ذخیره اطلاعات در دیتابیس!

۲. وظایف هفت‌گانه مهندسی نیازمندی‌ها (The ۷ Requirements Engineering Tasks)

مهندسی نیازمندی‌ها (Requirements Engineering - RE) یک فرآیند تکاملی و شامل ۷ فعالیت اصلی زیر است:

۲.۱. آغازین (Inception)

مفهوم اصلی: شروع پروژه با طرح سوالات بنیادی درباره اهداف کسب‌وکار، دامنه مسئله (Scope) و شناسایی ذینفعان اولیه‌ست.

- سوالات کلیدی: مشکل اصلی چیست؟ چه کسی ذینفع است؟ ارزش اقتصادی پروژه چقدر است؟
- مثال کاربردی: در سامانه بانکداری همراه (Mobile Banking)، در مرحله Inception مشخص می‌شود که آیا هدف «جذب مشتریان خرد» است یا «ارائه خدمات شارژ و قبوض».

۲.۲. استخراج (Elicitation)

مفهوم اصلی: کشف و استخراج خواسته‌ها از مشتریان، کاربران و صورت‌جلسات. در این مرحله ذینفعان اغلب نمی‌دانند دقیقاً چه می‌خواهند!

- تکنیک‌ها: مصاحبه، کارگاه‌های گروهی، مشاهده مستقیم (Observation) و پرسش‌نامه.
- مثال کاربردی: مشاهده رفتار انبارداران در یک شرکت لجستیک دیجی‌کالا جهت فهم نحوه اسکن بارکد کالاها در تاریکی انبار.

۲.۳. تفصیلی‌سازی (Elaboration)

مفهوم اصلی: خرد کردن اطلاعات خام استخراج‌شده و تبدیل آن‌ها به مدل‌های ساختاریافته (Use Caseها، دیاگرام‌های UML و مدل‌های داده).

- در این مرحله مشخص می‌شود چه کلاس‌ها، بازیگران (Actors) و جریان‌های کاری در سیستم وجود دارند.

۲.۴. مذاکره (Negotiation)

مفهوم اصلی: مشتریان خواسته‌های نامحدود دارند اما بودجه و زمان پروژه محدود است. مذاکره یعنی رسیدن به اولویت‌بندی مشترک و حل تعارضات.

- مدل برد-برد (Win-Win): مشتری عملکرد کلیدی را دریافت می‌کند و تیم توسعه زمان و بودجه منطقی را به دست می‌آورد.

۲.۵. تصریح و مستندسازی (Specification)

مفهوم اصلی: نگارش سند رسمی نیازمندی‌ها (SRS - Software Requirements Specification) یا ساخت کارت‌های داستان کاربر (User Stories) در متدولوژی‌های چابک.

۲.۶. اعتبارسنجی (Validation)

مفهوم اصلی: ارزیابی سند نیازمندی‌ها جهت اطمینان از عدم وجود تناقض، ابهام، ناتمامی یا غیرقابل‌تست بودن خواسته.

- بازبینی فنی (Technical Review): نشست کارشناسان برای بررسی سند قبل از شروع فاز طراحی.

۲.۷. مدیریت نیازمندی‌ها (Requirements Management)

مفهوم اصلی: ردیابی و کنترل تغییرات نیازمندی‌ها در طول چرخه حیات نرم‌افزار با استفاده از ماتریس ردیابی (Traceability Matrix).

۳. پایه‌ریزی کار: ذینفعان و زوایای دید متکثر (Stakeholders & Multiple Viewpoints)

ذینفع (Stakeholder) هر فرد یا سازمانی است که مستقیماً یا غیرمستقیم از سیستم سود می‌برد، زیان می‌بیند یا روی آن تاثیر می‌گذارد.

- دسته‌بندی ذینفعان: کاربران نهایی (End Users): کسانی که روزانه با سیستم کار می‌کنند (مثلاً اپراتور انبار).
- دسته‌بندی ذینفعان: مدیران کسب‌وکار (Business Managers): کسانی که سودآوری و بودجه برایشان مهم است.
- دسته‌بندی ذینفعان: تیم‌های پشتیبانی و نگهداری (Ops/Maintenance): کسانی که پایداری و لاگ‌گیری سیستم برایشان اولویت دارد.
- دسته‌بندی ذینفعان: نهادهای قانونی و رگولاتوری (Regulators): مانند قوانین بانک مرکزی یا سازمان بهداشت.

> 💡 تعارض زوایای دید (Multiple Viewpoints Example)
> در سامانه فروش بلیط هواپیما:
> • مسافر می‌خواهد خرید در ۱۰ ثانیه انجام شود (تسهیل کاربری).
> • مدیر مالی می‌خواهد تمام احراز هویت‌ها و تاییدیه‌های بانکی قبل از صدور انجام شود (امنیت و دقت).
> وظیفه مهندس نرم‌افزار، ایجاد موازنه بین این دیدگاه‌های متضاد است.

۴. نیازمندی‌های عملکردی (Functional) در برابر غیرعملکردی (Non-Functional / NFR)

یکی از مباحث فوق‌العاده مهم امتحانی و صنعتی، تفکیک دقیق این دو نوع نیازمندی است:

نوع نیازمندی	تعریف و چیستی	مثال در پروژه واقعی
عملکردی (Functional)	رفتار مستقیم سیستم، ورودی‌ها، خروجی‌ها و محاسباتی که سیستم باید انجام دهد.	«سامانه باید با وارد کردن کد ملی دانشجو، لیست دروس اخذشده را نمایش دهد.»
کارایی (Performance) (NFR)	سرعت، نرخ پاسخگویی و میزان مصرف منابع سیستم تحت بارگذاری.	«سیستم باید به ۱۰,۰۰۰ درخواست همزمان در کمتر از ۲ ثانیه پاسخ دهد.»
امنیت (Security) (NFR)	حفاظت از داده‌ها، رمزنگاری و سطوح دسترسی کاربران.	«تمام کلمه‌های عبور باید با الگوریتم Argon۲ رمزنگاری و ذخیره شوند.»
قابلیت اطمینان (Reliability NFR)	پایداری سیستم و عدم وقوع خرابی در بازه‌های زمانی مشخص.	«زمان آپ‌تایم سیستم پردازش پرداخت باید ۹۹.۹۹٪ در سال باشد.»

۵. مدلسازی سناریومحور و آناتومی Use Case

داستان کاربر (User Story) یا Use Case سناریویی است که تعامل یک بازیگر (Actor) با سیستم را برای رسیدن به یک هدف مشخص توصیف می‌کند.

- بازیگر اصلی (Primary Actor): فرد یا سیستمی که سناریو را برای رسیدن به هدفش آغاز می‌کند (مثلاً دانشجو).
- بازیگر فرعی (Secondary Actor): سیستمی که به ارائه خدمت کمک می‌کند (مثلاً درگاه پرداخت بانکی).

> 💡 آناتومی استاندارد یک Use Case (مثال امتحانی: رزرو اسیلوسکوپ در سیستم آزمایشگاه)
> • نام Use Case: رزرو تجهیزات آزمایشگاهی
> • بازیگر اصلی: دانشجو / استاد
> • پیش‌شرط (Precondition): کاربر در سامانه لاگین کرده و بدهی تاخیر ندارد.
> • جریان اصلی (Main Success Scenario):
> ۱. کاربر نام وسیله (اسیلوسکوپ) و تاریخ رزرو را وارد می‌کند.

۲. سیستم موجود بودن وسیله را در تاریخ مربوطه بررسی می‌کند.
۳. سیستم وضعیت وسیله را به «Reserved» تغییر داده و کد پیگیری صادر می‌کند.
• جریان‌های استثنا (Exceptions):
۲a. اگر وسیله در آن تاریخ رزرو بود، سیستم پیشنهادات زمانی جایگزین ارائه می‌دهد.

۶. قابلیت ردیابی نیازمندی‌ها (Requirements Traceability)

ماتریس ردیابی (Traceability Matrix) جدولی است که هر نیازمندی اولیه را به پروژه‌های طراحی، کلاس‌های کد و کیس‌های تست متصل می‌کند. فایده اصلی این ماتریس جلوگیری از خروج از محدوده (Scope Creep) و اطمینان از تست شدن تمام خواسته‌هاست.

۷. نمونه سوالات امتحانی و مفهومی فصل هفتم (همراه با پاسخ)

سوال ۱: چرا اضافه کردن مستقیم یک ویژگی به کد بدون طی مراحل مهندسی نیازمندی‌ها خطرناک است؟

پاسخ: زیرا باعث بروز پدیده Scope Creep و پنهان ماندن نیازمندی‌های غیرعملکردی (مثل امنیت و کارایی) می‌شود. همچنین هزینه اصلاح خطاهای ساختاری در فازهای بعدی تا ۱۰۰ برابر افزایش می‌یابد.

سوال ۲: تفاوت نیازمندی‌های Functional و Non-Functional را با یک مثال شفاف کنید.

پاسخ: نیازمندی عملکردی رفتار سیستم را بیان می‌کند (مثلاً: ثبت درخواست رزرو قطعه)؛ اما نیازمندی غیرعملکردی کیفیت اجرای آن رفتار را تعیین می‌کند (مثلاً: ثبت درخواست رزرو باید در کمتر از ۱ ثانیه انجام شود).

سوال ۳: مفهوم Win-Win Negotiation در مهندسی نیازمندی‌ها چیست؟

پاسخ: رویکردی که در آن تیم توسعه و مشتری به توافقی می‌رسند که هم اهداف کلیدی کسب‌وکار مشتری برآورده شود و هم زمان و بودجه پیشنهادی تیم توسعه منطقی و امکان‌پذیر باشد.

۸. راهنمای تکلیف هفتگی جهت کامیت در گیتهاب (فایل REQUIREMENTS.md)

دانشجویان عزیز باید برای پروژه اختصاصی خود در این ترم، فایلی به نام REQUIREMENTS.md در ریشه ریپازیتوری گیتهاب بسازند و موارد زیر را در آن کامیت کنند:

- ۱. لیست ذینفعان پروژه (حداقل ۳ نقش مختلف).
- ۲. ۵ نیازمندی عملکردی (Functional) و ۳ نیازمندی غیرعملکردی (NFR).
- ۳. نگارش کامل حداقل ۲ Use Case اصلی پروژه طبق آناتومی استاندارد.

فصل هشتم: مدلسازی نیازمندی‌ها (Requirements Modeling)

پیش‌گفتار: این جزوه آموزشی بر اساس فصل هشتم کتاب مرجع «مهندسی نرم‌افزار: رویکرد یک practitioner» نوشته پرسمن و مکسیم (ویرایش ۸ و ۹) تدوین شده است. در این فصل، پس از استخراج نیازمندی‌ها (فصل ۷)، وارد فاز مدلسازی تحلیل (Analysis Modeling) می‌شویم. مدل‌سازی پلی است میان استخراج نیازمندی‌ها و طراحی نرم‌افزار که دامنه مسئله را به سه دیدگاه اصلی «سناریومحور»، «کلاس‌محور» و «رفتاری» تقسیم می‌کند.

۱. فلسفه و اصول طلایی مدلسازی نیازمندی‌ها (Requirements Modeling Philosophy & Rules)

۱.۱. هدف از مدلسازی نیازمندی‌ها چیه؟

مدل تحلیل (Analysis Model) اولین بازنمایی فنی از نرم‌افزار است. در فاز استخراج نیازمندی‌ها (فصل ۷)، متون، گفتگوها و نیازمندی‌های کیفی به دست می‌آیند. اما طراحان و برنامه نویسان نمی‌توانند مستقیماً از روی متن‌های شلوغ کد بزنند! مدلسازی نیازمندی‌ها دو هدف کلیدی را دنبال می‌کند:

۱. ایجاد درک عمیق و شفاف از مسئله بدون درگیر شدن با جزئیات پیاده‌سازی (مثل دیتابیس یا زبان برنامه‌نویسی).
۲. ساخت پُلی استوار میان نیازمندی‌های مشتری و معماری نرم‌افزار.

۱.۲. سه قانون طلایی مدلسازی در کتاب پرسمن

- **قانون ۱: حفظ سطح انتزاع بالا (High Level of Abstraction):** مدل تحلیل باید بر روی «چه کاری باید انجام شود» (What) تمرکز کند، نه «چگونه انجام شود» (How). نباید در این فاز وارد ساختار دیتابیس، فرمت شبکه یا کدنویسی شوید.
- **قانون ۲: ساده‌سازی حداکثری و شفافیت (Keep it Simple):** مدل‌ها باید قابل فهم برای ذینفعان غیرفنی (مثل مدیران و کاربران) باشند. رسم نمودارهای بسیار پیچیده و شلوغ نقض غرض است.
- **قانون ۳: قابلیت ردیابی (Traceability):** هر عنصر موجود در مدل تحلیل (هر کلاس، Use Case یا حالت) باید مستقیماً به یک یا چند نیازمندی مطرح‌شده در فاز استخراج متصل باشد.

۲. مدلسازی سناریومحور (Scenario-Based Modeling)

مدلسازی سناریومحور دیدگاه سیستم را از چشم کاربران و بازیگران بیرونی (Actors) روایت می‌کند. سیستم‌های نرم‌افزاری وقتی موفق هستند که نیازهای عملیاتی کاربران را برطرف کنند.

۲.۱. Use Caseها و آناتومی استاندارد نگارش آن‌ها

یک Use Case متنی ساختاریافته است که تعامل یک بازیگر (Actor) با سیستم را برای دستیابی به یک هدف مشخص توصیف می‌کند. آکتور می‌تواند انسان (کاربر)، سیستم نرم‌افزاری دیگر یا یک سخت‌افزار باشد.

مثال واقعی کاربردی ۱: آناتومی Use Case در «سیستم مدیریت و رزرو تجهیزات آزمایشگاه»

- **نام Use Case:** رزرو دستگاه اسیلوسکوپ پیشرفته
- **آکتور اصلی (Primary Actor):** دانشجوی تحولات تکمیلی
- **پیش‌شرط‌ها (Preconditions):** دانشجو باید در سیستم لاگین کرده باشد و بدهی تاخیر نداشته باشد.
- **جریان اصلی (Main Success Scenario):** ۱. دانشجو نام دستگاه را جستجو می‌کند.
۲. سیستم موجودیت و تاریخ‌های خالی را نشان می‌دهد.
۳. دانشجو بازه زمانی (مثلاً ۲ روز) را انتخاب و درخواست ثبت می‌کند.
۴. سیستم درخواست را به صف تایید اپراتور آزمایشگاه می‌فرستد و کد پیگیری صادر می‌کند.
- **استثناها (Extensions):** ۳a. دستگاه در آن تاریخ رزرو است ◄ سیستم تاریخ‌های جایگزین پیشنهادی را نشان می‌دهد.

۲.۲. روابط بین Use Caseها: Include vs Extend

- **رابطه Include (داده‌شده/اجباری):** زمانی استفاده می‌شود که یک رفتار در چندین Use Case تکرار شود و اجباراً باید اجرا گردد. مثال در «سیستم بانکداری همراه»: Use Caseهای «انتقال کارت به کارت» و «پرداخت قبض» هر دو اجباراً شاملِ Use Case «احراز هویت کاربر» (Include) هستند.
- **رابطه Extend (توسعه/اختیاری):** زمانی استفاده می‌شود که یک رفتار فقط تحت شرایطی خاص و اختیاری به Use Case اصلی اضافه شود. مثال در «سیستم نوبت‌دهی آنلاین درمانگاه»: Use Case «رزرو نوبت پزشک» می‌تواند در صورت تمایل بیمار باِ Use Case «ارسال پیامک یادآوری» توسعه یابد (Extend).

۲.۳. نمودار فعالیت (Activity Diagram) و مسیرهای شناوری (Swimlanes)

نمودار فعالیت (Activity Diagram) جریان کاری (Workflow) یک فرآیند را از نقطه شروع تا پایان با نمایش تصمیم‌گیری‌ها و اجراهای موازی نشان می‌دهد. وقتی از مسیرهای شناوری (Swimlanes) استفاده می‌کنیم، صفحه به چند ستون تقسیم می‌شود تا مشخص گردد هر فعالیت توسط کدام نقش (کاربر، سیستم، دیتابیس) انجام می‌شود.

مثال واقعی کاربردی ۲: Activity Diagram با Swimlanes در «سامانه سفارش آنلاین غذا»

- **ستون مشتری:** انتخاب غذا ◄ ورود آدرس ◄ تایید نهایی سفارش
- **ستون سامانه:** بررسی موجودی ◄ محاسبه قیمت و تخفیف ◄ ارجاع به درگاه پرداخت

- **ستون آشپزخانه:** دریافت فاکتور ◄ آماده‌سازی غذا ◄ تحویل به پیک

۳. مدلسازی کلاس‌محور (Class-Based Modeling)

مدلسازی کلاس‌محور ساختار استاتیک و شی‌گرای سیستم را مشخص می‌کند. این مدل نشان می‌دهد چه اشیائی در سیستم وجود دارند، چه داده‌هایی را نگهداری می‌کنند و چه رفتارهایی دارند.

۳.۱. استخراج کلاس‌ها با تحلیل نحوی (Grammatical Parsing)

یکی از روش‌های استاندارد پرسمن برای کشف کلاس‌ها، بررسی متن نیازمندی‌ها است:
- اسم‌های عام (Nouns) ◄ کاندیدای کلاس یا ویژگی (Attribute) هستند.
- فعل‌ها (Verbs) ◄ کاندیدای متدها و رفتارها (Operations) هستند.

۳.۲. دسته‌بندی سه‌گانه کلاس‌ها در پرسمن

- **۱. کلاس‌های موجودیت (Entity Classes):** اطلاعات ماندگار در سیستم را نگهداری می‌کنند. (مثال در انبارداری: `Product`, `Supplier`, `Warehouse`).
- **۲. کلاس‌های مرزی (Boundary Classes):** رابط میان سیستم و دنیای بیرون هستند. (مثال: `LoginForm`, `BarcodeScannerInterface`).
- **۳. کلاس‌های کنترلی (Control Classes):** منطق کسب‌وکار و هماهنگی میان کلاس‌ها را مدیریت می‌کنند. (مثال: `OrderProcessor`, `PaymentGatewayManager`).

۳.۳. کارت‌های CRC (Class-Responsibility-Collaborator)

کارت CRC ابزاری فوق‌العاده ساده برای تفکیک مسئولیت‌ها است. روی یک برگه سه بخش تعریف می‌شود: نام کلاس، مسئولیت‌های کلاس (Responsibilities) و کلاس‌های همکار (Collaborators) که برای انجام مسئولیت به آن‌ها نیاز دارد.

مثال واقعی کاربردی ۳: کارت CRC برای کلاس `ReservationManager` در سیستم آزمایشگاه

- **نام کلاس:** ReservationManager (کلاس کنترلی)
- **مسئولیت‌ها (Responsibilities):** ۱. بررسی عدم تداخل زمانی رزرو جدید با رزروهای قبلی.
  ۲. محاسبه مبلغ جریمه تاخیر تحویل قطعه.
  ۳. تغییر وضعیت قطعه به Reserved.
- **همکاران (Collaborators):** کلاس `Equipment` ، کلاس `User` ، کلاس `FineCalculator`

۳.۴. نمودار کلاس (Class Diagram) و انواع روابط

- **ارتباط ساده Association) & Multiplicity):** ارتباط میان دو کلاس (مثلاً ۱ دانشجو می‌تواند ۰ تا چند رزرو داشته باشد: ۰..* to ۱).
- **تجمع ضعیف (Aggregation):** رابطه کل و جزء که در آن جزء می‌تواند مستقل از کل زنده بماند. (مثال: رابطه `Professor` و `Department` - اگر دپارتمان حذف شود، استاد هنوز وجود دارد).
- **تجمع قوی (Composition):** رابطه کل و جزء حیاتی که نابودی کل باعث نابودی جزء می‌شود. (مثال: `Invoice` و `InvoiceItem` - اگر فاکتور حذف شود، اقلام آن معنی ندارند).
- **وراثت (Inheritance / Generalization):** کلاس فرزند ویژگی‌های والد را به ارث می‌برد. (مثال: `Student` و `Professor` از `User` ارث می‌برند).

۴. مدلسازی رفتاری (Behavioral Modeling)

سیستم‌های نرم‌افزاری فقط ساختار ایستایی ندارند؛ آن‌ها در طول زمان به رویدادها (Events) پاسخ می‌دهند و وضعیت (State) خود را تغییر می‌دهند.

۴.۱. نمودار ماشین حالت (Statechart Diagram)

نمودار حالت چرخه حیات یک عنصر مهم سیستم را نشان می‌دهد که بر اثر رویدادهای مختلف از یک وضعیت به وضعیت دیگر منتقل می‌شود.

مثال واقعی کاربردی ۴: چرخه حیات حالت‌های قطعه آزمایشگاهی (`Equipment State`)

- **وضعیت Available (موجود):** رویداد ثبت درخواست ◄ انتقال به وضعیت Reserved (رزرو شده).
- **وضعیت Reserved (رزرو شده):** رویداد تحویل به دانشجو ◄ انتقال به وضعیت InUse (در حال استفاده).
- **وضعیت InUse (در حال استفاده):** رویداد بازگرداندن با خرابی ◄ انتقال به وضعیت Maintenance (در حال تعمیر).
- **وضعیت Maintenance (در حال تعمیر):** رویداد اتمام تعمیر ◄ بازگشت به وضعیت Available.

۴.۲. نمودار توالی (Sequence Diagram)

نمودار توالی (Sequence Diagram) تعاملات زمانی اشیاء را برای اجرای یک سناریوی مشخص نشان می‌دهد. محور عمودی خط زمان (Lifeline) و محور افقی اشیای درگیر در تعامل را نشان می‌دهد.

مثال واقعی کاربردی ۵: Sequence Diagram در «سامانه انتقال وجه همراه بانک»

- **ترتیب پیام‌ها:** ۱. `User` ── (ثبت شماره کارت و مبلغ) ──► `MobileUI`
۲. `MobileUI` ── (CheckBalance) ──► `AccountManager`
۳. `AccountManager` ── (DeductMoney) ──► `AccountDatabase`
۴. `MobileUI` ◄── (TransferSuccess) ── `AccountManager` ── (نمایش رسید) ──► `User`

۵. مدلسازی چابک و الگوهای تحلیل (Agile Modeling & Analysis Patterns)

- **مدلسازی چابک (Agile Modeling):** در متدولوژی‌های چابک (مثل اسکرام)، مدلسازی تعطیل نمی‌شود! بلکه مدل‌ها به صورت سریع، انتزاعی و روی وایت‌برد رسم می‌شوند تا فهم مشترک ایجاد کنند، سپس کدنویسی انجام می‌شود.
- **الگوهای تحلیل (Analysis Patterns):** راهکارهای آزموده‌شده برای مسائل متداول در مدلسازی. به جای اختراع دوباره چرخ، از الگوهای استانداردی مثل الگوی `Party-Role` (برای مدیریت افراد و سازمان‌ها) استفاده می‌شود.

۶. جدول مقایسه جامع مدل‌های سه‌گانه تحلیل

جدول زیر دیدگاه‌های سه‌گانه مدلسازی نیازمندی‌ها را بر اساس ابزارها، هدف و ابعاد مقایسه می‌کند:

دیدگاه مدلسازی	نمودارها و ابزارهای اصلی	تمرکز اصلی (Focus)	مثال کاربردی صنعتی
سناریومحور (Scenario-Based)	Use Case Diagram, Activity Diagram	روایت تعامل کاربر با سیستم و جریان کاری	فرآیند سفارش غذا در اسنپ‌فود
کلاس‌محور (Class-Based)	Class Diagram, CRC Cards	ساختار استاتیک اشیاء، داده‌ها و روابط	مدیریت موجودی و اقلام در انبار دیجی‌کالا
رفتاری (Behavioral)	Statechart Diagram, Sequence Diagram	تغییر حالت‌ها و تعاملات زمانی اشیاء	چرخه تراکنش مالی و انتقال وجه در همراه بانک

۷. نمونه سوالات امتحانی تشریحی به همراه پاسخ کلیدی

سوال ۱: تفاوت رابطه Include و Extend در نمودار Use Case چیست؟ با ذکر یک مثال روشن توضیح دهید.

پاسخ کلیدی: رابطه Include نشان‌دهنده یک رفتار اجباری و مشترک است که همیشه اجرا می‌شود (مانند احراز هویت در بانکداری). اما رابطه Extend نشان‌دهنده یک رفتار اختیاری است که تنها تحت شرایط خاصی به Use Case اصلی اضافه می‌شود (مانند دریافت رسید کاغذی در عابر بانک).

سوال ۲: کارت CRC چیست و سه بخش اصلی آن کدامند؟

پاسخ کلیدی: کارت CRC ابزاری ساده برای مدلسازی کلاس‌محور است. سه بخش آن عبارتند از: ۱. نام کلاس (Class Name). ۲ مسئولیت‌ها (Responsibilities). ۳ همکاران (Collaborators) که کلاس برای انجام وظایفش به آن‌ها متکی است.

سوال ۳: چه زمانی از نمودار ماشین حالت (Statechart) و چه زمانی از نمودار توالی (Sequence) استفاده می‌کنیم؟

پاسخ کلیدی: زمانی از Statechart استفاده می‌کنیم که بخواهیم چرخه حیات و تغییر وضعیت‌های «یک شیء مهم» (مثل تغییر وضعیت قطعه آزمایشگاهی) را بررسی کنیم. اما از Sequence Diagram زمانی استفاده می‌شود که بخواهیم تعاملات زمانی «چندین شیء مختلف» را در اجرای یک سناریو مشخص نشان دهیم.

۸. راهنمای تکالیف عملی و کامیت گیتهاب (فایل `MODELING.md`)

دانشجویان محترم باید تا قبل از جلسه آینده، مستندات مدلسازی پروژه انتخابی خود را در قالبی تحت عنوان `MODELING.md` در ریپازیتوری گیتهاب شخصی خود کامیت (Commit) و ارسال (Push) کنند. سرفصل‌های الزامی این فایل عبارتند از:

۱. رسم نمودار Use Case اصلی سیستم همراه با سناریوی متنی ۱ Use Case کلیدی.

۲. استخراج حداقل ۴ کلاس اصلی و نگارش ۱ کارت CRC برای کلاس کنترلی پروژه.

۳. رسم نمودار کلاس (Class Diagram) و مشخص کردن نوع روابط.

۴. رسم یک نمودار حالت (Statechart) یا نمودار توالی (Sequence) برای فرآیند اصلی سیستم.

فصل نهم: مفاهیم و اصول پایه طراحی نرم‌افزار (Design Concepts)

طراحی نرم‌افزار (Software Design) هسته فنی مهندسی نرم‌افزار است. اگر فاز تحلیل نیازمندی‌ها (Requirements Analysis) پاسخ به سوال «سیستم چه کاری باید انجام دهد؟» (What) باشد، فاز طراحی پاسخی صریح و مهندسی به سوال «سیستم چگونه باید ساخته شود؟» (How) است.

به بیان روجر پرسمن، کیفیت نرم‌افزار در فاز طراحی شکل می‌گیرد و نه در فاز کدنویسی. کدنویسی تنها ترجمه مدل طراحی به یک زبان برنامه‌نویسی است. بدون یک طراحی اصولی، نرم‌افزار دچار بدهی فنی، ناتوانی در تغییر و شکنندگی ساختاری خواهد شد.

۱.۱. تبدیل مدل تحلیل به مدل طراحی

در فاز طراحی، ۴ لایه اصلی از مدل تحلیل استخراج و بازآفرینی می‌شوند:

- ۱. طراحی معماری (Architectural Design): نگاشت عناصر ساختاری و ساختار کلی ماژول‌ها.
- ۲. طراحی کامپوننت/مفهومی (Component-Level Design): تبدیل نمودارهای UML و کلاس‌ها به ساختار کدهای اجرایی و الگوریتم‌ها.
- ۳. طراحی واسط کاربر (User Interface Design): طراحی نحوه تعامل انسان و سیستم و ارتباطات خارجی.
- ۴. طراحی داده/داده‌ساختار (Data/Class Design): تبدیل نمودارهای کلاس و ERD به مدل‌های داده‌ای دیتابیس یا ساختارهای حافظه.

۲. مفاهیم بنیادی و اصول طلایی طراحی

پرسمن مجموعه‌ای از مفاهیم کلیدی را معرفی می‌کند که شالوده تمام روش‌های طراحی نرم‌افزار در صنعت هستند:

۲.۱. انتزاع (Abstraction)

انتزاع یعنی تمرکز بر روی ویژگی‌های حیاتی یک عنصر و چشم‌پوشی از جزئیات غیرضروری در سطوح بالای منطق. انتزاع در دو شکل اصلی ظاهر می‌شود:

• الف) انتزاع رویه‌ای (Procedural Abstraction): مجموعه‌ای از دستورات که یک وظیفه مشخص را انجام می‌دهند بدون آنکه جزئیات گام‌های آن در سطح بالا افشا شود (مانند تابع ()CalculateTax).

• ب) انتزاع داده‌ای (Data Abstraction): مجموعه‌ای از داده‌ها که یک شیء را توصیف می‌کنند همراه با اعمال مجاز روی آن‌ها (مانند کلاس BankAccount).

۲.۲. معماری نرم‌افزار (Software Architecture)

معماری، ساختار یکپارچه سیستم، نحوه سازمان‌دهی اجزا (Components)، روابط میان آن‌ها و الگوهای تعاملی حاکم بر سیستم است.

۲.۳. الگوی طراحی (Design Patterns)

یک الگوی طراحی، راه‌حلی آزموده‌شده، استاندارد و بازاستفاده‌پذیر برای یک مسأله رایج و تکرارشونده در یک بافت خاص از طراحی نرم‌افزار است (مانند الگوی Singleton یا Observer).

۲.۴. جداسازی دغدغه‌ها (Separation of Concerns)

هر مسأله پیچیده نرم‌افزاری اگر به بخش‌های مستقل و کوچک‌تر شکسته شود، ساده‌تر حل می‌شود. یک «دغدغه» (Concern) می‌تواند یک ویژگی عملکردی، امنیت، یا کارایی باشد. با جداسازی دغدغه‌ها، تغییر در یک بخش اثر منفی روی بخش‌های دیگر نمی‌گذارد.

۲.۵. پیمانه‌ای بودن (Modularity) و منحنی هزینه

پیمانه‌ای بودن یعنی تقسیم نرم‌افزار به ماژول‌ها یا نام‌های مستقل (مثل توابع، کلاس‌ها، پکیج‌ها). پرسمن چالش پیمانه‌ای بودن را با منحنی هزینه تحلیل می‌کند:

• • پدیده زیرپیمانه‌ای (Under-Modularity): اگر تعداد ماژول‌ها خیلی کم باشد، اندازه هر ماژول بسیار بزرگ شده و هزینه درک و تغییر آن سرسام‌آور می‌شود.

• • پدیده بیش‌پیمانه‌ای (Over-Modularity): اگر نرم‌افزار بیش از حد به ماژول‌های کوچک تقسیم شود، هزینه ارتباط و یکپارچه‌سازی (Integration/Interconnection) میان ماژول‌ها صعودی می‌شود.

• • نقطه بهینه: نقطه‌ای در منحنی هزینه که مجموع هزینه توسعه ماژول‌ها و هزینه یکپارچه‌سازی آن‌ها کمینه (Minimum) می‌شود.

۲.۶. پنهان‌سازی اطلاعات (Information Hiding)

این اصل که توسط دیوید پارناس (David Parnas) مطرح شد، بیان می‌کند که هر ماژول باید به گونه‌ای طراحی شود که جزئیات الگوریتم‌ها و ساختار داده‌های داخلی آن از دید سایر ماژول‌ها پنهان بماند. دسترسی به ماژول فقط از طریق واسط‌های مشخص (Interfaces) انجام می‌شود. این اصل از انتشار خطاهایی که بر اثر تغییرات کدهای داخلی رخ می‌دهند جلوگیری می‌کند.

۳. استقلال عملکردی: همبستگی (Cohesion) و تزویج (Coupling)

استقلال عملکردی (Functional Independence) مهم‌ترین معیار ارزیابی کیفیت طراحی یک ماژول است. استقلال با دو شاخص اندازه‌گیری می‌شود: همبستگی (Cohesion) و تزویج (Coupling). شعار طلایی مهندسی نرم‌افزار عبارت است از: «همبستگی بالا (High Cohesion) و تزویج پایین (Low Coupling)».

۳.۱. همبستگی (Cohesion) و سطوح مختلف آن

همبستگی یعنی میزان پیوستگی و ارتباط درونی عناصر داخل یک ماژول. ماژولی که همبستگی بالا دارد، فقط و فقط یک وظیفه مشخص را به طور کامل انجام می‌دهد.

پرسمن سطوح همبستگی را از ضعیف‌ترین (بدترین) تا قوی‌ترین (بهترین) به شرح زیر دسته‌بندی می‌کند:

- ۱. همبستگی تصادفی (Coincidental Cohesion - بدترین نوع): عناصری در یک ماژول قرار گرفته‌اند که هیچ ارتباط منطقی با هم ندارند (مانند یک کلاس Helper کشکول با توابع بی‌ربط).
- ۲. همبستگی منطقی (Logical Cohesion): عناصری که کارهای مشابهی انجام می‌دهند در یک ماژول جمع شده‌اند اما بر اساس ورودی انتخاب می‌شوند (مانند تابعی که بر اساس یک الگوریتم سوئیچ، هم فایل چاپ می‌کند هم گزارش ایمیل می‌فرستد).
- ۳. همبستگی زمانی (Temporal Cohesion): توابعی که در یک زمان مشخص اجرا می‌شوند در یک جا قرار گرفته‌اند (مانند تابع InitializeSystem() که هم دیتابیس را باز می‌کند هم متغیرهای گرافیکی را مقداردهی اولیه می‌کند).
- ۴. همبستگی روشی (Procedural Cohesion): توابعی که باید به یک ترتیب خاص پشت سر هم اجرا شوند در یک ماژول قرار می‌گیرند.
- ۵. همبستگی ارتباطی (Communicational Cohesion): توابعی که روی داده‌های ورودی یا خروجی یکسان کار می‌کنند در یک ماژول قرار گرفته‌اند.
- ۶. همبستگی توالی (Sequential Cohesion): خروجی یک بخش از ماژول، ورودی مستقیم بخش بعدی همان ماژول است.

• ۷. همبستگی عملکردی (Functional Cohesion - بهترین نوع): تمام عناصر ماژول برای انجام دقیقاً «یک وظیفه مشخص و واحد» همکاری می‌کنند (مانند تابع CalculateInterest Rate()).

۳.۲. تزویج (Coupling) و سطوح مختلف آن

تزویج یعنی میزان وابستگی و اتکای میان ماژول‌های مختلف. ما می‌خواهیم تزویج تا حد امکان پایین (Low Coupling) باشد تا تغییر در یک ماژول باعث خرابی ماژول‌های دیگر نشود.

انواع تزویج از قوی‌ترین/بدترین به ضعیف‌ترین/بهترین:

• ۱. تزویج محتوایی (Content Coupling - بدترین نوع): یک ماژول مستقیماً داده‌ها یا کدهای داخلی ماژول دیگر را دستکاری می‌کند (نقض کامل Information Hiding).

• ۲. تزویج عمومی (Global/Common Coupling): چندین ماژول به یک متغیر عمومی (Global Variable) یا حافظه مشترک دسترسی داشته و آن را تغییر می‌دهند.

• ۳. تزویج کنترلی (Control Coupling): یک ماژول با ارسال یک پرچم کنترلی (Flag) به ماژول دیگر، نحوه اجرا و منطق آن را کنترل می‌کند.

• ۴. تزویج ساختار داده (Stamp/Data-Structured Coupling): یک ساختار داده کامل (مثل یک Record یا Object) به ماژولی پاس داده می‌شود در حالی که آن ماژول فقط به یک فیلد کوچک آن نیاز دارد.

• ۵. تزویج داده‌ای (Data Coupling - بهترین نوع): ماژول‌ها فقط با ارسال پارامترهای ساده و ضروری داده‌ای با هم ارتباط برقرار می‌کنند.

جدول مقایسه جامع همبستگی (Cohesion) و تزویج (Coupling)

معیار مقایسه	همبستگی (Cohesion)	تزویج (Coupling)	هدف مهندسی نرم‌افزار
تعریف بنیادی	ارتباط و پیوستگی عناصر داخل یک ماژول	میزان وابستگی و اتکای بین ماژول‌های متفاوت	بررسی کیفیت ساختاری ماژول‌ها
وضعیت مطلوب	حداکثر ممکن (High Cohesion)	حداقل ممکن (Low Coupling)	رسیدن به استقلال عملکردی کامل
اثر بر نگه‌داری	افزایش شدید خوانایی و سهولت تست واحد	کاهش اثر دومینویی خطاها در زمان تغییر	کاهش هزینه توسعه و توسعه چابک

بهترین نوع	همبستگی عملکردی (Functional)	تزویج داده‌ای (Data Coupling)	ساخت سیستم‌های مقیاس‌پذیر

۴. بازنویسی کد (Refactoring) و کلاس‌های طراحی

۴.۱. بازنویسی کد (Refactoring)

بازنویسی کد یعنی تغییر و بهبود ساختار داخلی نرم‌افزار بدون تغییر در رفتار بیرونی آن. هدف از Refactoring عبارت است از:

- ساده‌سازی کدهای پیچیده و اصلاح کدهای دارای «بوی بد» (Code Smells).
- افزایش همبستگی و کاهش تزویج ماژول‌ها.
- کاهش بدهی فنی (Technical Debt) و آماده‌سازی سیستم برای تغییرات جدید.

۴.۲. انواع کلاس‌های طراحی (Design Classes)

در مرحله طراحی، کلاس‌های مدل تحلیل به ۵ دسته کلاس عملیاتی تبدیل می‌شوند:

- ۱. کلاس‌های واسط کاربر (User Interface Classes): مدیریت فرم‌ها، پنجره‌ها و تعامل با کاربر.
- ۲. کلاس‌های دامنه کسب‌وکار (Business Domain Classes): پیاده‌سازی قوانین اصلی منطق تجاری سیستم.
- ۳. کلاس‌های فرآیندی (Process Classes): مدیریت جریان‌های کاری و مدیریت فرآیندهای پیچیده.
- ۴. کلاس‌های ماندگاری (Persistent Classes): مدیریت ذخیره‌سازی داده‌ها در دیتابیس یا فایل‌ها.
- ۵. کلاس‌های سیستم (System Classes): مدیریت امکانات سطح پایین مثل شبکه، امنیت و سخت‌افزار.

۵. بررسی مثال‌های کاربردی در پروژه‌های مختلف صنعتی

مثال ۱: سیستم بانکداری متمرکز (Core Banking) — پنهان‌سازی اطلاعات

در طراحی سامانه محاسبه سود سپرده‌ها، الگوریتم‌های پیچیده مالی داخل کلاس `InterestCalculator` پنهان می‌شوند. سایر بخش‌های سیستم (مثل داشبورد موبایل‌بانک) فقط متد `calculateInterest(accountNumber)` را فراخوانی می‌کنند و هیچ اطلاعی از فرمول‌های داخلی دیتابیس یا نحوه محاسبه ندارند (Information Hiding).

مثال ۲: نرم‌افزار پردازش تصویر پزشکی (MRI Scanner) — انتزاع و همبستگی عملکردی

ماژول `DicomImageProcessor` دارای همبستگی عملکردی (Functional Cohesion) است؛ زیرا تمامی توابع آن صرفاً روی نویزگیری و فیلترگذاری تصویر فشرده MRI کار می‌کنند. هیچ تابع مربوط به نمایش گرافیکی یا ثبت نام بیمار در این ماژول وجود ندارد.

مثال ۳: اپلیکیشن تاکسی آنلاین (اسنپ/تپسی) — تزویج داده‌ای (Data Coupling)

سرویس تخصیص راننده (DriverMatcher) و سرویس محاسبه قیمت سفر (FareCalculator) با هم تزویج داده‌ای دارند؛ به این صورت که سرویس قیمت‌گذاری فقط دو متغیر ساده `(origin, destination)` را دریافت کرده و یک عدد Decimal برمی‌گرداند بدون آنکه نیازی به ساختار داخلی شیء راننده داشته باشد.

مثال ۴: سیستم مدیریت و رزرو تجهیزات آزمایشگاه — بازنویسی کد (Refactoring)

در نسخه اول سیستم، کلاس `EquipmentManager` هم کار ثبت وسایل را انجام می‌داد، هم محاسبه جریمه دیرکرد، هم ارسال ایمیل یادآوری (همبستگی تصادفی/منطقی ضعیف). با Refactoring، این کلاس به ۳ کلاس با مسئولیت واحد شکسته‌شد: `FineCalculator`، `EquipmentRepository` و `NotificationService` (رسیدن به High Cohesion).

۶. نمونه سوالات امتحانی تشریحی به همراه پاسخ کلیدی

سوال ۱: چرا همبستگی بالا (High Cohesion) و تزویج پایین (Low Coupling) ایده آل مهندسی نرم‌افزار است؟

پاسخ: همبستگی بالا باعث می‌شود هر ماژول تمرکز کامل روی یک وظیفه مشخص داشته باشد که این امر خوانایی، تست واحد (Unit Testing) و بازاستفاده‌پذیری کد را افزایش می‌دهد. تزویج پایین وابستگی ماژول‌ها به یکدیگر را کمینه‌می‌کند؛ در نتیجه تغییر یا بروز خطا در یک ماژول اثر دومینویی روی سایر بخش‌های سیستم نخواهد گذاشت.

سوال ۲: مفهوم پنهان‌سازی اطلاعات (Information Hiding) را توضیح داده و فایده آن را بنویسید.

پاسخ: پنهان‌سازی اطلاعات به این معناست که جزئیات الگوریتم‌های داخلی و ساختار داده‌های یک ماژول نباید برای سایر ماژول‌ها قابل مشاهده باشد. فایده اصلی آن این است که اگر منطق داخلی یا ساختار دیتابیس یک ماژول تغییر کند، هیچ نیازی به تغییر در کدهای ماژول‌های فراخواننده نخواهد بود.

سوال ۳: پدیده Under-modularity و Over-modularity را بر اساس منحنی هزینه پیمانه‌ای بودن توضیح دهید.

پاسخ: در Under-modularity تعداد ماژول‌ها بسیار کم و حجم آن‌ها بسیار بزرگ است که باعث افزایش شدید هزینه توسعه و درک کد می‌شود. در Over-modularity نرم‌افزار بیش از حد خرد شده که باعث افزایش صعودی هزینه یکپارچه‌سازی و ارتباط میان ماژول‌ها می‌گردد. هدف مهندسی نرم‌افزار رسیدن به نقطه کمینه مجموع این دو هزینه است.

۷. راهنمای تکلیف گیت‌هاب جهت کامیت پروژه‌های دانشجویی

دانشجویان موظفند فایل `DESIGN_CONCEPTS.md` را در ریپازیتوری شخصی پروژه خود ساخته و موارد زیر را مستند کنند:

- ۱. دسته‌بندی اصلی ماژول‌های پروژه خود به همراه لایه‌بندی معماری.
- ۲. شناسایی یک نمونه از همبستگی (Cohesion) و تزویج (Coupling) در کدهای پروژه و تحلیل سطح آن.
- ۳. ارائه یک نمونه سناریوی Refactoring که روی پروژه خود انجام داده‌اند تا کد تمیزتر شود.

فصل ۱۰ و ۱۱: طراحی معماری و طراحی در سطح کامپونننت (Architectural & Component-Level Design)

پیش‌گفتار

فصل ۱۵ و ۱۶: مفاهیم کیفیت نرم‌افزار و بازبینی‌های فنی (Quality Concepts & Software Reviews)

پیشگفتار

فصل ۱۹ و ۲۰: استراتژی‌ها و تکنیک‌های تست نرم‌افزار (در سطح مؤلفه و ادغام)

پیش

فصل ۲۲: مدیریت پیکربندی نرم‌افزار (Software Configuration Management - SCM)

پیش

فصل ۲۴ و ۲۶: مفاهیم مدیریت پروژه و تحلیل و مدیریت ریسک

پیش

فصل ۲۵: برنامه‌ریزی و تخمین پروژه نرم‌افزاری | فصل ۲۷: نگهداری، تکامل و مهندسی مجدد

پیشگفتار